Complete System Analysis (Formatted)

```
Complete Frontend Directory Analysis
1. Complete File Tree
d:\Portal\frontend\
■■■ adminPages/                          # Admin Portal (Vite + React SPA)
■   ■■■ .env.example
■   ■■■ .gitignore
■   ■■■ metadata.json
■   ■■■ package.json
■   ■■■ README.md
■   ■■■ payloads.md
■   ■■■ server.ts                        # Express server with placeholder API
■   ■■■ tsconfig.json
■   ■■■ vite.config.ts
■   ■■■ index.html
■   ■■■ src/
■       ■■■ main.tsx                     # React entry point
■       ■■■ App.tsx                      # Root component (696 lines, single-file app)
■       ■■■ index.css                    # Tailwind theme (Luminous design system)
■
■■■ candidatePages/                      # Candidate Portal (Next.js App Router)
■   ■■■ .gitignore
■   ■■■ AGENTS.md                        # Next.js agent rules note
■   ■■■ API_ENDPOINTS.md                 # Full API spec (1534+ lines)
■   ■■■ CLAUDE.md                        # Next.js version warning
■   ■■■ README.md
■   ■■■ eslint.config.mjs
■   ■■■ next.config.ts
■   ■■■ package.json
■   ■■■ package-lock.json
■   ■■■ postcss.config.mjs
■   ■■■ tsconfig.json
■   ■■■ public/
■   ■   ■■■ favicon.ico
■   ■   ■■■ file.svg
■   ■   ■■■ globe.svg
■   ■   ■■■ next.svg
■   ■   ■■■ vercel.svg
■   ■   ■■■ window.svg
■   ■■■ src/
■       ■■■ app/
■       ■   ■■■ favicon.ico
■       ■   ■■■ globals.css              # Tailwind theme + custom utilities
■       ■   ■■■ layout.tsx               # Root layout (Manrope + Inter fonts)
■       ■   ■■■ page.tsx                 # Landing/home page
■       ■   ■■■ application-success/
■       ■   ■   ■■■ page.tsx             # Success confirmation page
■       ■   ■■■ dashboard/
■       ■   ■   ■■■ candidate/
■       ■   ■       ■■■ page.tsx         # Candidate dashboard
■       ■   ■■■ jobs/
■       ■   ■   ■■■ page.tsx             # Job listings with filters
■       ■   ■■■ knowledge-factory/
■       ■   ■   ■■■ page.tsx             # Knowledge Factory application form
■       ■   ■■■ smart-match/
■       ■       ■■■ page.tsx             # AI resume upload + matching
■       ■■■ components/
■           ■■■ Header.tsx               # Shared navigation header
■           ■■■ Footer.tsx               # Shared footer
■
■■■ recruitersPage/                      # Recruiter Portal (Vite + React SPA)
    ■■■ .env.example
    ■■■ .gitignore
    ■■■ metadata.json
    ■■■ package.json
    ■■■ package-lock.json
    ■■■ README.md
    ■■■ server.ts                        # Express server with placeholder API
    ■■■ test-payloads.json               # Test payloads for API endpoints
    ■■■ tsconfig.json
    ■■■ vite.config.ts
    ■■■ index.html
    ■■■ src/
```

```
        ■■■ main.tsx                    # React entry point
        ■■■ App.tsx                     # Router setup (react-router-dom)
        ■■■ index.css                   # Tailwind theme (Fluid design system)
        ■■■ vite-env.d.ts
        ■■■ components/
        ■   ■■■ Layout.tsx              # Sidebar + top nav shell
        ■   ■■■ ui/
        ■       ■■■ Button.tsx          # Reusable button component
        ■       ■■■ Card.tsx            # Reusable card component with variants
        ■■■ data/
        ■   ■■■ applications.csv         # Seed data: 10 candidate applications
        ■   ■■■ jobs.csv                 # Seed data: 6 job listings
        ■   ■■■ recruiterSeed.ts         # CSV parser + typed data + metrics
        ■■■ lib/
        ■   ■■■ utils.ts                # cn() utility (clsx + tailwind-merge)
        ■■■ pages/
            ■■■ Dashboard.tsx           # Recruiter dashboard with metrics
            ■■■ Jobs.tsx                # Job management with search/filter
            ■■■ Applications.tsx        # Candidate application review table
            ■■■ CreateJob.tsx           # Job creation form
            ■■■ BulkUpload.tsx          # Resume bulk upload interface
```

2. Tech Stack Summary

Portal■Framework■Router■Build Tool■CSS■Styling Approach
candidatePages■Next.js 16.2.2■Next.js App Router (file-based)■Next.js built-in■Tailwind CSS v4■Cust
recruitersPage■React 19■react-router-dom v7■Vite 6■Tailwind CSS v4■Fluid design system, soul-gradie
adminPages■React 19■None (single-page, state-based views)■Vite 6■Tailwind CSS v4■Luminous design sy
Common across all three:

TypeScript 5.x
React 19
Tailwind CSS v4 (with @tailwindcss/vite or @tailwindcss/postcss)
Google Material Symbols Outlined icon font (candidate) / Lucide React icons (recruiter + admin)
Motion/motion-react for animations (recruiter + admin)
Gemini AI integration (@google/genai in recruiter + admin)

3. Pages/Components Breakdown

Candidate Portal (Next.js App Router)

Route■File■Purpose
/■src/app/page.tsx■Landing page: hero, trust indicators, stats, how-it-works, benefits, CTA
/jobs■src/app/jobs/page.tsx■Job listings page with sidebar filters (location, role type), hardcoded
/smart-match■src/app/smart-match/page.tsx■AI resume upload + match results (hardcoded 3 matches with
/knowledge-factory■src/app/knowledge-factory/page.tsx■Application form for Knowledge Factory program
/dashboard/candidate■src/app/dashboard/candidate/page.tsx■Candidate dashboard: active application pr
/application-success■src/app/application-success/page.tsx■Success confirmation after applying

Shared Components:

src/components/Header.tsx – Sticky glass-nav with logo, nav links (Home, Jobs, Dashboard, Knowledge
src/components/Footer.tsx – 5-column footer: Brand, Platform links, Resources, Legal, social icons

Recruiter Portal (Vite + react-router-dom)

Route■File■Purpose
/■src/pages/Dashboard.tsx■Metrics overview: pipeline volume, under review, AI score, shortlisted, re
/jobs■src/pages/Jobs.tsx■Job listings table with search, department filter, status filter, CRUD acti
/applications■src/pages/Applications.tsx■Candidate applications table with AI match scores, search,
/create-job■src/pages/CreateJob.tsx■Multi-section form: job details, description editor, skills mana
/bulk-upload■src/pages/BulkUpload.tsx■Drag-and-drop resume upload with file status tracking

Shared Components:

src/components/Layout.tsx – Fixed sidebar (72px) + sticky top nav (glass effect), nav items: Dashboa
src/components/ui/Button.tsx – Variants: primary/secondary/tertiary/ghost; Sizes: sm/md/lg; supports
src/components/ui/Card.tsx – Variants: lowest/low/high/highest (surface hierarchy backgrounds)

Admin Portal (Vite, single-file SPA)

Everything is in src/App.tsx (696 lines) with internal state-based view switching:

View■Component (internal)■Purpose
Portal view■AdminPortal■Stats grid (live performance chart, AI accuracy), User Management table, Job
Add HR view■AddHRView■HR account creation form + existing HR personnel table
Internal sub-components: TopNav, StatsGrid, UserManagement, JobManagement, EmailAutomations, AddHRVi

4. Data Files and Schemas

recruitersPage/src/data/jobs.csv

```
id,title,department,location,status,applicants,newToday,postedDate,timeToHireDays
1,Senior UX Designer,Design,"San Francisco, CA",Active,128,12,2026-03-29,21
2,Product Manager,Product,Remote,Active,84,4,2026-03-24,19
3,Lead Backend Engineer,Engineering,"New York, NY",Draft,0,0,,
4,Growth Marketing Lead,Marketing,"London, UK",Closed,246,0,2026-02-10,27
5,Data Analyst,Data,Remote,Active,61,3,2026-04-02,16
6,Talent Operations Partner,People,"Austin, TX",Closed,39,0,2026-01-18,14
```

recruitersPage/src/data/applications.csv

```
id,name,email,phone,role,match,status,date,avatarSeed
1,Elena Soros,elena.s@example.com,+1 (555) 012-3456,Senior UX Designer,94,Shortlisted,2026-04-07,ele
2,Marcus Johnson,marcus.j@example.com,+1 (555) 012-7890,Senior UX Designer,78,Pending,2026-04-06,mar
...
(10 total rows)
recruitersPage/src/data/recruiterSeed.ts
This is the critical data layer. It:

Parses both CSV files at build time using a custom CSV parser (handles quoted fields)
Exports typed interfaces: RecruiterJob, RecruiterApplication, RecruiterJobStatus, RecruiterApplicatio
Maps job departments to Lucide icons (Design=LayoutGrid, Marketing=TrendingUp, People=Users, default=
Generates avatar URLs from picsum.photos using seed values
Computes recruiterMetrics aggregate object:
totalJobs, activeJobs, totalApplicants, newApplications
averageTimeToHireDays, totalApplications
shortlistedApplications, pendingApplications, rejectedApplications
averageMatchScore, shortlistRate
recruitersPage/test-payloads.json
Defines example payloads for:

Jobs create/update
Applications submit/updateStatus
Bulk upload files
5. API Endpoint Configurations
Candidate Portal – API_ENDPOINTS.md (the definitive spec, 1534+ lines)
All endpoints use /api/v1 prefix. Key endpoints documented:

Authentication: POST /api/v1/auth/register, /login, /refresh, /logout, /forgot-password, /reset-passw

Profiles: GET|PUT /api/v1/profiles/me (candidate), GET|PUT /api/v1/profiles/recruiter/me

Jobs: GET /api/v1/jobs (search/filter), GET /api/v1/jobs/:jobId, POST /api/v1/jobs, PUT /api/v1/jobs/

Applications: POST /api/v1/applications, GET /api/v1/applications/me, GET /api/v1/applications/:id, P

Resume/Smart Match: POST /api/v1/resume/upload, GET /api/v1/matches, GET /api/v1/resume/:id/parse-res

Knowledge Factory: GET /api/v1/knowledge-factory/programs, GET /api/v1/knowledge-factory/programs/:id

Recruiter: GET /api/v1/recruiter/pipeline

Recruiter Server (server.ts) – Express on port 3000
All endpoints are placeholders (return empty data with TODO comments):

GET|POST /api/jobs, GET|PUT|DELETE /api/jobs/:id
GET|POST /api/applications, GET|PUT /api/applications/:id
POST /api/bulk-upload
Admin Server (server.ts) – Express on port 3000
Also placeholders:

GET|POST /api/users, PATCH|DELETE /api/users/:id
GET|POST /api/jobs, PATCH|DELETE /api/jobs/:id
POST /api/automations/email
Critical observation: Both servers run on the same port (3000). The candidate Next.js app also defaul

6. Portal Separation Architecture
The three portals are completely separate applications with independent:

Aspect■Candidate■Recruiter■Admin
Framework■Next.js 16■React 19 + Vite■React 19 + Vite
Routing■File-based (app/)■react-router-dom■State-based (useState<View>)
Server■Next.js built-in■Express + Vite middleware■Express + Vite middleware
Dependencies■next, react, react-dom■react-router-dom, express, lucide-react, motion■express, motion,
Design Tokens■Unique (Manrope+Inter)■Unique (Inter only)■Unique (Inter only, teal accent)
Data Source■Hardcoded inline data■CSV files + TypeScript parser■Hardcoded inline state
Package management■Independent node_modules■Independent node_modules■Independent node_modules
No shared code exists between the three portals. Each has its own package.json, node_modules, CSS the

7. Routing Configuration
Candidate (Next.js App Router)
File-based routing:

/ --> src/app/page.tsx
/jobs --> src/app/jobs/page.tsx
/smart-match --> src/app/smart-match/page.tsx
```

/knowledge-factory --> src/app/knowledge-factory/page.tsx
/dashboard/candidate --> src/app/dashboard/candidate/page.tsx
/application-success --> src/app/application-success/page.tsx
Root layout wraps all pages via src/app/layout.tsx
Recruiter (react-router-dom)
Defined in src/App.tsx:

/   --> Dashboard
/jobs   --> Jobs
/applications --> Applications
/create-job   --> CreateJob
/bulk-upload --> BulkUpload
*   --> Dashboard (fallback)
All routes are wrapped in the Layout component (sidebar + top nav).

Admin (state-based)
No routing library. Uses a View type ('portal' | 'add-hr') and useState to switch between views. Navi

8. Key Shared vs. Duplicated Functionality
Duplicated (not shared):
Design systems: Each portal has its own Tailwind theme with overlapping but distinct color tokens. Al
Job management: All three portals have job-related UI (candidate browses, recruiter manages, admin ma
Application handling: Candidate submits, recruiter reviews, admin has no direct application UI but ha
Header/Navigation: Candidate has its own Header/Footer; recruiter has Layout with sidebar+topnav; adm
Express servers: Recruiter and admin both have server.ts with overlapping /api/jobs and /api/applicat
What should be shared but is not:
No shared UI component library (Button, Card exist only in recruiter)
No shared API client or endpoint constants (candidate has API_ENDPOINTS.md as documentation only, not
No shared TypeScript types/interfaces for API payloads
No shared CSV parsing logic
No shared authentication logic
No shared validation schemas
9. Build Configuration Highlights
Candidate (candidatePages/package.json)
dev: next dev
build: next build
start: next start
Recruiter (recruitersPage/package.json)
dev: tsx server.ts   (runs Express server with Vite middleware)
build: vite build
start: node server.ts   (production: serves dist/ statically)
Admin (adminPages/package.json)
dev: tsx server.ts
build: vite build
All three use TypeScript with tsc --noEmit for linting. Both Vite apps load environment variables fro

10. Summary of Key Findings
Three completely independent apps with zero code sharing, each in its own directory with separate dep
All data is currently static/mock – candidate pages hardcode jobs and matches inline; recruiter uses
API endpoints are documented but not implemented – the Express servers are skeletons with TODO comme
Port conflict risk – all three default to port 3000.
The API_ENDPOINTS.md file in the candidate portal is the most comprehensive artifact, defining a full
Design inconsistency – while all three share a “Coastal Seven” brand and teal color palette, each has
other
Agent: Analyze resume_service backend
Output
I now have a comprehensive understanding of the entire service. Here is the complete analysis:

Complete Analysis of `d:\Portal\resume_service`

1. API ENDPOINTS

The service is a FastAPI application (`main.py`) exposing these endpoints:

GET `/health`
- **Purpose**: Liveness probe
...

Thinking
I’ll conduct a thorough analysis of the resume_service directory. Let me start by mapping the directo

Search
ListFiles: resume_service
■

```
18 lines of output
Search
Glob: '**/*.py' in path 'd:\Portal\resume_service'
■
25 lines of output
Search
Glob: '**/*.{txt,toml,cfg,ini,yaml,yml,json,env,md,rst}' in path 'd:\Portal\resume_service'
■
18 lines of output

Thinking
Now let me read all the core Python files in parallel:

ReadFile

main.py
from __future__ import annotations

import time
import uuid

from fastapi import FastAPI, HTTPException, Request
from fastapi.responses import JSONResponse

from .extractors import ExtractionError, UnsupportedResumeError
from .multipart_parser import MultipartError, parse_multipart
from .observability import METRICS, RATE_LIMITER, log_event
from .parser import parse_resume_document
from .schemas import CandidateType, ErrorResponse, ScoreResumeRequest
from .scoring import score_resume
from .storage import init_storage, list_parsed_resumes, load_parsed_resume, save_parsed_resume

app = FastAPI(
    title="Resume Service",
    version="0.1.0",
    description=(
        "Internal deterministic resume parsing and scoring service for Indian resume normalization. "
        "Use `/v1/parse` for multipart file uploads and `/v1/score` for JD scoring."
    ),
)

@app.on_event("startup")
async def startup() -> None:
    init_storage()

@app.middleware("http")
async def request_context_middleware(request: Request, call_next):
    request_id = request.headers.get("x-request-id") or str(uuid.uuid4())
    request.state.request_id = request_id

    route_key = request.url.path
    if route_key in {"/v1/parse", "/v1/score"}:
        client_ip = request.client.host if request.client else "unknown"
        if not RATE_LIMITER.allow(client_ip, route_key):
            log_event(request_id=request_id, endpoint=route_key, status=429, failure_reason="rate_lim
            return JSONResponse(status_code=429, content=ErrorResponse(message="Rate limit exceeded.

    start_time = time.perf_counter()
    response = await call_next(request)
    response.headers["x-request-id"] = request_id

    latency_ms = (time.perf_counter() - start_time) * 1000
    if route_key == "/v1/parse" and response.status_code < 400:
        METRICS.parse_success_count += 1
        METRICS.parse_latency_total_ms += latency_ms
    elif route_key == "/v1/score" and response.status_code < 400:
        METRICS.score_request_count += 1
        METRICS.score_latency_total_ms += latency_ms
    return response

@app.exception_handler(MultipartError)
async def multipart_error_handler(request: Request, exc: MultipartError) -> JSONResponse:
    if request.url.path == "/v1/parse":
        METRICS.parse_failure_count += 1
    log_event(
        request_id=getattr(request.state, "request_id", None),
```

```
            endpoint=request.url.path,
            status=400,
            failure_reason=str(exc),
        )
        return JSONResponse(status_code=400, content=ErrorResponse(message=str(exc)).model_dump())

    @app.exception_handler(UnsupportedResumeError)
    async def unsupported_resume_handler(request: Request, exc: UnsupportedResumeError) -> JSONResponse:
        if request.url.path == "/v1/parse":
            METRICS.parse_failure_count += 1
        message = str(exc)
        if "unsupported" in message.lower():
            METRICS.unsupported_format_reject_count += 1
        if "scanned" in message.lower() or "image-based" in message.lower():
            METRICS.scanned_resume_reject_count += 1
        log_event(
            request_id=getattr(request.state, "request_id", None),
            endpoint=request.url.path,
            status=422,
            failure_reason=message,
        )
        return JSONResponse(status_code=422, content=ErrorResponse(message=str(exc)).model_dump())

    @app.exception_handler(ExtractionError)
    async def extraction_error_handler(request: Request, exc: ExtractionError) -> JSONResponse:
        if request.url.path == "/v1/parse":
            METRICS.parse_failure_count += 1
        log_event(
            request_id=getattr(request.state, "request_id", None),
            endpoint=request.url.path,
            status=400,
            failure_reason=str(exc),
        )
        return JSONResponse(status_code=400, content=ErrorResponse(message=str(exc)).model_dump())

    @app.exception_handler(HTTPException)
    async def http_exception_handler(request: Request, exc: HTTPException) -> JSONResponse:
        if request.url.path == "/v1/parse" and exc.status_code >= 400:
            METRICS.parse_failure_count += 1
        log_event(
            request_id=getattr(request.state, "request_id", None),
            endpoint=request.url.path,
            status=exc.status_code,
            failure_reason=exc.detail,
        )
        if isinstance(exc.detail, str):
            return JSONResponse(status_code=exc.status_code, content={"detail": exc.detail})
        return JSONResponse(status_code=exc.status_code, content=exc.detail)

    @app.get("/health")
    async def healthcheck() -> dict[str, str]:
        return {"status": "ok"}

    @app.get("/metrics")
    async def metrics_endpoint():
        return METRICS.snapshot().model_dump()

    @app.post(
        "/v1/parse",
        summary="Parse a resume",
        openapi_extra={
            "requestBody": {
                "required": True,
                "content": {
                    "multipart/form-data": {
                        "schema": {
                            "type": "object",
                            "required": ["file", "resume_id"],
                            "properties": {
                                "file": {
                                    "type": "string",
                                    "format": "binary",
                                    "description": "Resume file in PDF or DOCX format.",
                                },
                                "resume_id": {
                                    "type": "string",
                                    "description": "Resume identifier from the main backend.",
```

```
                                "example": "res_123",
                            },
                            "candidate_type": {
                                "type": "string",
                                "enum": ["fresher", "lateral"],
                                "default": "lateral",
                            },
                        },
                    }
                }
            },
        }
    },
)
async def parse_resume_endpoint(request: Request):
    parts = parse_multipart(await request.body(), request.headers.get("content-type"))
    file_part = parts.get("file")
    resume_id_part = parts.get("resume_id")
    candidate_type_part = parts.get("candidate_type")

    if not file_part or not file_part.filename:
        raise HTTPException(status_code=422, detail="Multipart field 'file' is required.")
    if not resume_id_part or not resume_id_part.text:
        raise HTTPException(status_code=422, detail="Multipart field 'resume_id' is required.")

    try:
        candidate_type = CandidateType(candidate_type_part.text) if candidate_type_part and candidate
    except ValueError as exc:
        raise HTTPException(status_code=422, detail="Multipart field 'candidate_type' must be 'fresh
    response = parse_resume_document(
        file_bytes=file_part.data,
        filename=file_part.filename,
        resume_id=resume_id_part.text,
        candidate_type=candidate_type,
        content_type=file_part.content_type,
    )
    save_parsed_resume(response)
    log_event(
        request_id=getattr(request.state, "request_id", None),
        endpoint="/v1/parse",
        status=200,
        resume_id=response.resume_id,
        extractor=response.parser_metadata.extractor_name,
        detected_columns=response.parser_metadata.detected_columns,
    )
    return response.model_dump()

@app.post(
    "/v1/score",
    summary="Score a parsed resume against a job description",
    description="Use stored parsed resume data by sending only `resume_id` and `job_context`, or over
)
async def score_resume_endpoint(request: Request, request_body: ScoreResumeRequest):
    if request_body.normalized_resume is None or request_body.resume_text is None:
        stored_resume = load_parsed_resume(request_body.resume_id)
        if stored_resume is None:
            raise HTTPException(
                status_code=404,
                detail="Parsed resume not found for this resume_id. Call /v1/parse first or provide r
            )
        request_body = ScoreResumeRequest(
            resume_id=request_body.resume_id,
            normalized_resume=stored_resume.normalized_resume,
            resume_text=stored_resume.resume_text,
            job_context=request_body.job_context,
        )

    response = score_resume(request_body)
    log_event(
        request_id=getattr(request.state, "request_id", None),
        endpoint="/v1/score",
        status=200,
        resume_id=request_body.resume_id,
        recommendation=response.recommendation,
    )
    return response.model_dump()
```

```
@app.get("/v1/resumes")
async def list_resumes_endpoint(page: int = 1, page_size: int = 20):
    safe_page = max(1, page)
    safe_page_size = min(100, max(1, page_size))
    return list_parsed_resumes(page=safe_page, page_size=safe_page_size).model_dump()

@app.get("/v1/resumes/{resume_id}")
async def get_resume_endpoint(resume_id: str):
    stored_resume = load_parsed_resume(resume_id)
    if stored_resume is None:
        raise HTTPException(status_code=404, detail="Parsed resume not found for this resume_id.")
    return {
        "resume_id": stored_resume.resume_id,
        "candidate_type": stored_resume.candidate_type.value,
        "normalized_resume": stored_resume.normalized_resume.model_dump(),
        "resume_quality": stored_resume.resume_quality.model_dump(),
        "parser_metadata": stored_resume.parser_metadata.model_dump(),
        "resume_text": stored_resume.resume_text,
        "duplicate_of_resume_id": stored_resume.duplicate_of_resume_id,
        "created_at": stored_resume.created_at,
        "updated_at": stored_resume.updated_at,
    }
▼ Show more
ReadFile

schemas.py
from __future__ import annotations

from enum import Enum
from typing import Any

from pydantic import BaseModel, ConfigDict, Field, field_validator, model_validator

class CandidateType(str, Enum):
    fresher = "fresher"
    lateral = "lateral"

class EmploymentRecord(BaseModel):
    company: str | None = None
    title: str | None = None
    start_date: str | None = None
    end_date: str | None = None
    employment_type: str | None = None
    location: str | None = None
    description: str | None = None
    is_current: bool = False

class EducationRecord(BaseModel):
    degree: str | None = None
    specialization: str | None = None
    institution: str | None = None
    completion_year: int | None = None
    score: str | None = None

class ProjectRecord(BaseModel):
    title: str
    tech_stack: list[str] = Field(default_factory=list)
    summary: str | None = None
    source_section: str

class CertificationRecord(BaseModel):
    name: str
    issuer: str | None = None
    issued_date: str | None = None

class SkillEvidenceRecord(BaseModel):
    skill: str
    evidence_type: str
    source_section: str

class NormalizedResume(BaseModel):
    full_name: str | None = None
    emails: list[str] = Field(default_factory=list)
    phones: list[str] = Field(default_factory=list)
    current_location: str | None = None
    links: list[str] = Field(default_factory=list)
```

```
    skills: list[str] = Field(default_factory=list)
    skill_evidence: list[SkillEvidenceRecord] = Field(default_factory=list)
    experience_summary: str | None = None
    employment_history: list[EmploymentRecord] = Field(default_factory=list)
    education: list[EducationRecord] = Field(default_factory=list)
    projects: list[ProjectRecord] = Field(default_factory=list)
    certifications: list[CertificationRecord] = Field(default_factory=list)
    current_role: str | None = None
    full_time_months: int = 0
    internship_months: int = 0
    total_relevant_months: int = 0
    notice_period: str | None = None
    current_ctc: str | None = None
    expected_ctc: str | None = None
    preferred_location: str | None = None
    work_authorization: str | None = None

class ParserMetadata(BaseModel):
    filename: str
    file_type: str
    content_type: str | None = None
    file_size_bytes: int
    line_count: int
    page_count: int = 1
    extractor_name: str | None = None
    section_order: list[str] = Field(default_factory=list)
    detected_columns: int = 1
    is_scanned: bool = False
    content_fingerprint: str | None = None
    sensitive_findings: list[str] = Field(default_factory=list)
    extracted_text: str

class ParseResumeResponse(BaseModel):
    resume_id: str
    candidate_type: CandidateType
    normalized_resume: NormalizedResume
    warnings: list[str] = Field(default_factory=list)
    missing_fields: list[str] = Field(default_factory=list)
    resume_quality: "ResumeQualityResult"
    parser_metadata: ParserMetadata

class JobContext(BaseModel):
    job_id: str | None = None
    title: str | None = None
    description: str | None = None
    required_skills: list[str] = Field(default_factory=list)
    preferred_skills: list[str] = Field(default_factory=list)
    keywords: list[str] = Field(default_factory=list)
    experience_min_years: float | None = None
    experience_max_years: float | None = None
    location: str | None = None

class ScoreBreakdown(BaseModel):
    jd_match: dict[str, float]
    resume_quality: dict[str, float]

class ResumeQualityResult(BaseModel):
    score: int
    breakdown: dict[str, float]
    recommendation: str
    summary: str | None = None

class ScoreResumeRequest(BaseModel):
    resume_id: str
    normalized_resume: NormalizedResume | None = None
    resume_text: str | None = None
    job_context: JobContext

    model_config = ConfigDict(
        json_schema_extra={
            "example": {
                "resume_id": "res_123",
                "job_context": {
                    "job_id": "job_backend_intern_001",
                    "title": "Backend Developer Intern",
                    "description": "Looking for Python, SQL, Git, Flask or FastAPI, and Docker exposu
```

```
                    "required_skills": ["Python", "SQL", "Git"],
                    "preferred_skills": ["Flask", "Docker", "AWS"],
                    "keywords": ["backend", "api", "internship", "problem solving"],
                    "experience_min_years": 0,
                    "experience_max_years": 1,
                    "location": "Hyderabad",
                },
            }
        }
    )

    @field_validator("resume_text")
    @classmethod
    def ensure_resume_text(cls, value: str | None) -> str | None:
        if value is None:
            return value
        cleaned = value.strip()
        if not cleaned:
            raise ValueError("resume_text must not be empty")
        return cleaned

    @model_validator(mode="after")
    def ensure_resume_inputs_are_complete(self) -> "ScoreResumeRequest":
        has_resume = self.normalized_resume is not None
        has_text = self.resume_text is not None
        if has_resume != has_text:
            raise ValueError("Provide both normalized_resume and resume_text together, or omit both f
        return self

class ScoreResumeResponse(BaseModel):
    resume_id: str
    jd_match_score: int
    resume_quality_score: int
    breakdown: ScoreBreakdown
    matched_skills: list[str] = Field(default_factory=list)
    missing_skills: list[str] = Field(default_factory=list)
    recommendation: str
    warnings: list[str] = Field(default_factory=list)
    summary: str | None = None

class ResumeListItem(BaseModel):
    resume_id: str
    candidate_type: CandidateType
    full_name: str | None = None
    current_role: str | None = None
    filename: str | None = None
    extractor_name: str | None = None
    page_count: int = 1
    content_fingerprint: str | None = None
    duplicate_of_resume_id: str | None = None
    created_at: str
    updated_at: str

class ResumeListResponse(BaseModel):
    page: int
    page_size: int
    total: int
    items: list[ResumeListItem] = Field(default_factory=list)

class MetricsResponse(BaseModel):
    parse_success_count: int = 0
    parse_failure_count: int = 0
    score_request_count: int = 0
    unsupported_format_reject_count: int = 0
    scanned_resume_reject_count: int = 0
    average_parse_latency_ms: float = 0.0
    average_score_latency_ms: float = 0.0

class ErrorResponse(BaseModel):
    status: str = "error"
    message: str
    detail: dict[str, Any] | None = None
```

▼ Show more

ReadFile

storage.py

```
from __future__ import annotations

import json
import sqlite3
import shutil
from dataclasses import dataclass
from datetime import datetime, timezone
from pathlib import Path
import re
from threading import Lock
from uuid import uuid4

from .schemas import (
    CandidateType,
    NormalizedResume,
    ParseResumeResponse,
    ParserMetadata,
    ResumeListItem,
    ResumeListResponse,
    ResumeQualityResult,
)

DATA_DIR = Path(__file__).resolve().parent / "data"
RESUME_STORE_DIR = DATA_DIR / "resumes"
DB_PATH = DATA_DIR / "resume_catalog.sqlite3"
_STORAGE_READY = False
_DB_LOCK = Lock()
_CONNECTION: sqlite3.Connection | None = None

@dataclass(slots=True)
class StoredResume:
    resume_id: str
    candidate_type: CandidateType
    normalized_resume: NormalizedResume
    resume_text: str
    parser_metadata: ParserMetadata
    resume_quality: ResumeQualityResult
    duplicate_of_resume_id: str | None = None
    created_at: str | None = None
    updated_at: str | None = None

def init_storage() -> None:
    global _STORAGE_READY
    if _STORAGE_READY:
        return
    DATA_DIR.mkdir(parents=True, exist_ok=True)
    RESUME_STORE_DIR.mkdir(parents=True, exist_ok=True)
    try:
        _initialize_database()
    except sqlite3.OperationalError:
        _reset_database_files()
        _initialize_database()
    _STORAGE_READY = True

def save_parsed_resume(response: ParseResumeResponse) -> None:
    init_storage()
    now = _utc_now()
    fingerprint = response.parser_metadata.content_fingerprint

    with _DB_LOCK:
        connection = _connect()
        duplicate_of_resume_id = _resolve_duplicate_resume_id(connection, response.resume_id, fingerp
        connection.execute(
            """
            INSERT INTO parsed_resumes (
                resume_id,
                candidate_type,
                normalized_resume_json,
                parser_metadata_json,
                resume_quality_json,
                resume_text,
                filename,
                full_name,
                current_role,
                content_fingerprint,
                duplicate_of_resume_id,
```

```
                    created_at,
                    updated_at
                ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
                ON CONFLICT(resume_id) DO UPDATE SET
                    candidate_type = excluded.candidate_type,
                    normalized_resume_json = excluded.normalized_resume_json,
                    parser_metadata_json = excluded.parser_metadata_json,
                    resume_quality_json = excluded.resume_quality_json,
                    resume_text = excluded.resume_text,
                    filename = excluded.filename,
                    full_name = excluded.full_name,
                    current_role = excluded.current_role,
                    content_fingerprint = excluded.content_fingerprint,
                    duplicate_of_resume_id = excluded.duplicate_of_resume_id,
                    updated_at = excluded.updated_at
                """,
                (
                    response.resume_id,
                    response.candidate_type.value,
                    json.dumps(response.normalized_resume.model_dump()),
                    json.dumps(response.parser_metadata.model_dump()),
                    json.dumps(response.resume_quality.model_dump()),
                    response.parser_metadata.extracted_text,
                    response.parser_metadata.filename,
                    response.normalized_resume.full_name,
                    response.normalized_resume.current_role,
                    fingerprint,
                    duplicate_of_resume_id,
                    now,
                    now,
                ),
            )
            _reconcile_duplicate_chain(connection, fingerprint)
            connection.commit()

    def load_parsed_resume(resume_id: str) -> StoredResume | None:
        init_storage()
        with _DB_LOCK:
            connection = _connect()
            row = connection.execute(
                "SELECT * FROM parsed_resumes WHERE resume_id = ?",
                (resume_id,),
            ).fetchone()
        if row is None:
            return None
        return _stored_resume_from_row(row)

    def list_parsed_resumes(*, page: int, page_size: int) -> ResumeListResponse:
        init_storage()
        offset = max(0, (page - 1) * page_size)
        with _DB_LOCK:
            connection = _connect()
            total = int(connection.execute("SELECT COUNT(*) FROM parsed_resumes").fetchone()[0])
            rows = connection.execute(
                """
                SELECT resume_id, candidate_type, full_name, current_role, filename, content_fingerprint,
                       duplicate_of_resume_id, created_at, updated_at, parser_metadata_json
                FROM parsed_resumes
                ORDER BY updated_at DESC, resume_id DESC
                LIMIT ? OFFSET ?
                """,
                (page_size, offset),
            ).fetchall()

        items: list[ResumeListItem] = []
        for row in rows:
            parser_metadata = ParserMetadata.model_validate(json.loads(row["parser_metadata_json"]))
            items.append(
                ResumeListItem(
                    resume_id=row["resume_id"],
                    candidate_type=CandidateType(row["candidate_type"]),
                    full_name=row["full_name"],
                    current_role=row["current_role"],
                    filename=row["filename"],
                    extractor_name=parser_metadata.extractor_name,
                    page_count=parser_metadata.page_count,
                    content_fingerprint=row["content_fingerprint"],
```

```
                duplicate_of_resume_id=row["duplicate_of_resume_id"],
                created_at=row["created_at"],
                updated_at=row["updated_at"],
            )
        )
    return ResumeListResponse(page=page, page_size=page_size, total=total, items=items)

def migrate_legacy_json_store(*, retire_legacy_files: bool = False) -> dict[str, int | str]:
    init_storage()
    run_id = uuid4().hex
    started_at = _utc_now()
    status = "running"
    imported = 0
    skipped = 0
    failed = 0
    retired = 0
    files = sorted(RESUME_STORE_DIR.glob("*.json"))
    error_message: str | None = None

    _start_migration_run(
        run_id=run_id,
        started_at=started_at,
        source_dir=str(RESUME_STORE_DIR),
        retire_legacy_files=retire_legacy_files,
        files_discovered=len(files),
    )

    try:
        for path in files:
            try:
                stored = json.loads(path.read_text(encoding="utf-8"))
                response = ParseResumeResponse(
                    resume_id=stored["resume_id"],
                    candidate_type=CandidateType(stored["candidate_type"]),
                    normalized_resume=NormalizedResume.model_validate(stored["normalized_resume"]),
                    warnings=stored.get("warnings", []),
                    missing_fields=stored.get("missing_fields", []),
                    resume_quality=ResumeQualityResult.model_validate(stored["resume_quality"]),
                    parser_metadata=ParserMetadata.model_validate(stored["parser_metadata"]),
                )
            except Exception:
                failed += 1
                continue

            existing = load_parsed_resume(response.resume_id)
            if existing is not None:
                skipped += 1
                continue

            save_parsed_resume(response)
            imported += 1

            if retire_legacy_files:
                _retire_legacy_json(path, run_id)
                retired += 1
        status = "success"
    except Exception as exc:
        status = "failed"
        error_message = str(exc)
    finally:
        finished_at = _utc_now()
        _finish_migration_run(
            run_id=run_id,
            finished_at=finished_at,
            status=status,
            imported=imported,
            skipped=skipped,
            failed=failed,
            retired=retired,
            error_message=error_message,
        )

    return {
        "run_id": run_id,
        "status": status,
        "files_discovered": len(files),
```

```
        "imported_count": imported,
        "skipped_count": skipped,
        "failed_count": failed,
        "retired_count": retired,
    }

def load_migration_run(run_id: str) -> dict[str, str | int] | None:
    init_storage()
    with _DB_LOCK:
        row = _connect().execute("SELECT * FROM migration_runs WHERE run_id = ?", (run_id,)).fetchone
    if row is None:
        return None
    return dict(row)

def resume_store_path(resume_id: str) -> Path:
    safe_id = re.sub(r"[^A-Za-z0-9._-]", "_", resume_id).strip("._") or "resume"
    return RESUME_STORE_DIR / f"{safe_id}.json"

def _connect() -> sqlite3.Connection:
    global _CONNECTION
    if _CONNECTION is None:
        _CONNECTION = sqlite3.connect(DB_PATH, timeout=30, check_same_thread=False)
        _CONNECTION.row_factory = sqlite3.Row
    return _CONNECTION

def _stored_resume_from_row(row: sqlite3.Row) -> StoredResume:
    return StoredResume(
        resume_id=row["resume_id"],
        candidate_type=CandidateType(row["candidate_type"]),
        normalized_resume=NormalizedResume.model_validate(json.loads(row["normalized_resume_json"])),
        parser_metadata=ParserMetadata.model_validate(json.loads(row["parser_metadata_json"])),
        resume_quality=ResumeQualityResult.model_validate(json.loads(row["resume_quality_json"])),
        resume_text=row["resume_text"],
        duplicate_of_resume_id=row["duplicate_of_resume_id"],
        created_at=row["created_at"],
        updated_at=row["updated_at"],
    )

def _utc_now() -> str:
    return datetime.now(timezone.utc).isoformat()

def _initialize_database() -> None:
    with _DB_LOCK:
        connection = _connect()
        connection.execute("PRAGMA journal_mode=MEMORY")
        connection.executescript(
            """
            CREATE TABLE IF NOT EXISTS parsed_resumes (
                resume_id TEXT PRIMARY KEY,
                candidate_type TEXT NOT NULL,
                normalized_resume_json TEXT NOT NULL,
                parser_metadata_json TEXT NOT NULL,
                resume_quality_json TEXT NOT NULL,
                resume_text TEXT NOT NULL,
                filename TEXT,
                full_name TEXT,
                current_role TEXT,
                content_fingerprint TEXT,
                duplicate_of_resume_id TEXT,
                created_at TEXT NOT NULL,
                updated_at TEXT NOT NULL
            );

            CREATE TABLE IF NOT EXISTS migration_runs (
                run_id TEXT PRIMARY KEY,
                started_at TEXT NOT NULL,
                finished_at TEXT,
                status TEXT NOT NULL,
                source_dir TEXT NOT NULL,
                retire_legacy_files INTEGER NOT NULL DEFAULT 0,
                files_discovered INTEGER NOT NULL DEFAULT 0,
                imported_count INTEGER NOT NULL DEFAULT 0,
                skipped_count INTEGER NOT NULL DEFAULT 0,
                failed_count INTEGER NOT NULL DEFAULT 0,
                retired_count INTEGER NOT NULL DEFAULT 0,
                error_message TEXT
```

```
            );
            """
        )
        connection.commit()

def _resolve_duplicate_resume_id(connection: sqlite3.Connection, resume_id: str, fingerprint: str | N
    if not fingerprint:
        return None

    row = connection.execute(
        """
        SELECT resume_id, duplicate_of_resume_id
        FROM parsed_resumes
        WHERE content_fingerprint = ? AND resume_id <> ?
        ORDER BY created_at ASC, resume_id ASC
        LIMIT 1
        """,
        (fingerprint, resume_id),
    ).fetchone()
    if row is None:
        return None
    return row["duplicate_of_resume_id"] or row["resume_id"]

def _reconcile_duplicate_chain(connection: sqlite3.Connection, fingerprint: str | None) -> None:
    if not fingerprint:
        return

    rows = connection.execute(
        """
        SELECT resume_id
        FROM parsed_resumes
        WHERE content_fingerprint = ?
        ORDER BY created_at ASC, resume_id ASC
        """,
        (fingerprint,),
    ).fetchall()
    if len(rows) <= 1:
        return

    canonical_resume_id = rows[0]["resume_id"]
    connection.execute(
        "UPDATE parsed_resumes SET duplicate_of_resume_id = NULL WHERE resume_id = ?",
        (canonical_resume_id,),
    )
    connection.execute(
        """
        UPDATE parsed_resumes
        SET duplicate_of_resume_id = ?
        WHERE content_fingerprint = ? AND resume_id <> ?
        """,
        (canonical_resume_id, fingerprint, canonical_resume_id),
    )

def _retire_legacy_json(path: Path, run_id: str) -> None:
    retired_dir = RESUME_STORE_DIR.parent / "resumes_retired" / run_id
    retired_dir.mkdir(parents=True, exist_ok=True)
    destination = retired_dir / path.name
    shutil.move(str(path), str(destination))

def _start_migration_run(
    *,
    run_id: str,
    started_at: str,
    source_dir: str,
    retire_legacy_files: bool,
    files_discovered: int,
) -> None:
    with _DB_LOCK:
        connection = _connect()
        connection.execute(
            """
            INSERT OR REPLACE INTO migration_runs (
                run_id,
                started_at,
                status,
                source_dir,
```

```
                retire_legacy_files,
                files_discovered
            ) VALUES (?, ?, ?, ?, ?, ?)
            """,
            (run_id, started_at, "running", source_dir, 1 if retire_legacy_files else 0, files_discov
        )
        connection.commit()

def _finish_migration_run(
    *,
    run_id: str,
    finished_at: str,
    status: str,
    imported: int,
    skipped: int,
    failed: int,
    retired: int,
    error_message: str | None,
) -> None:
    with _DB_LOCK:
        connection = _connect()
        connection.execute(
            """
            UPDATE migration_runs
            SET finished_at = ?,
                status = ?,
                imported_count = ?,
                skipped_count = ?,
                failed_count = ?,
                retired_count = ?,
                error_message = ?
            WHERE run_id = ?
            """,
            (finished_at, status, imported, skipped, failed, retired, error_message, run_id),
        )
        connection.commit()

def _reset_database_files() -> None:
    global _CONNECTION
    if _CONNECTION is not None:
        try:
            _CONNECTION.close()
        except Exception:
            pass
        _CONNECTION = None
    for suffix in ("", "-journal", "-wal", "-shm"):
        stale_path = Path(f"{DB_PATH}{suffix}")
        if stale_path.exists():
            try:
                stale_path.unlink(missing_ok=True)
            except PermissionError:
                pass
▼ Show more
ReadFile

scoring.py
from __future__ import annotations

import re
from collections import OrderedDict

from .constants import SKILL_ALIASES
from .normalization import estimate_total_experience_months
from .schemas import JobContext, ResumeQualityResult, ScoreBreakdown, ScoreResumeRequest, ScoreResume
from .settings import SETTINGS

def score_resume(request: ScoreResumeRequest) -> ScoreResumeResponse:
    resume_skills = _canonicalize_skills(request.normalized_resume.skills)
    job_required_skills = _canonicalize_skills(request.job_context.required_skills)
    job_preferred_skills = _canonicalize_skills(request.job_context.preferred_skills)
    keywords = _job_keywords(request.job_context)
    evidence_strength = _skill_evidence_strength(request.normalized_resume.skill_evidence)

    matched_required = sorted(set(resume_skills) & set(job_required_skills))
    missing_required = sorted(set(job_required_skills) - set(resume_skills))
    matched_preferred = sorted(set(resume_skills) & set(job_preferred_skills))
```

```
    required_strength = sum(evidence_strength.get(skill, 1.0) for skill in matched_required)
    preferred_strength = sum(evidence_strength.get(skill, 1.0) for skill in matched_preferred)
    skills_ratio = 1.0 if not job_required_skills else required_strength / len(job_required_skills)
    preferred_ratio = 1.0 if not job_preferred_skills else preferred_strength / len(job_preferred_ski
    skills_component = round(
        SETTINGS.jd_score_weights.skills * ((skills_ratio * 0.85) + (preferred_ratio * 0.15)),
        2,
    )

    experience_years = _weighted_experience_years(request)
    experience_component = round(
        SETTINGS.jd_score_weights.experience * _experience_alignment(experience_years, request.job_co
        2,
    )

    keyword_component = round(
        SETTINGS.jd_score_weights.keywords * _keyword_overlap_ratio(keywords, request.resume_text, re
        2,
    )
    jd_match_score = int(round(skills_component + experience_component + keyword_component))

    quality_result = score_resume_quality(
        resume_id=request.resume_id,
        normalized_resume=request.normalized_resume,
        resume_text=request.resume_text,
    )
    completeness_component = quality_result.breakdown["completeness"]
    structure_component = quality_result.breakdown["structure"]
    contact_component = quality_result.breakdown["contactability"]
    signal_component = quality_result.breakdown["signal_quality"]
    resume_quality_score = quality_result.score

    warnings: list[str] = []
    if missing_required:
        warnings.append("Some required job skills are missing from the normalized resume.")
    if request.normalized_resume.notice_period and "immediate" not in request.normalized_resume.notic
        warnings.append(f"Candidate notice period detected: {request.normalized_resume.notice_period

    return ScoreResumeResponse(
        resume_id=request.resume_id,
        jd_match_score=jd_match_score,
        resume_quality_score=resume_quality_score,
        breakdown=ScoreBreakdown(
            jd_match={
                "skills": skills_component,
                "experience": experience_component,
                "keywords": keyword_component,
            },
            resume_quality={
                "completeness": completeness_component,
                "structure": structure_component,
                "contactability": contact_component,
                "signal_quality": signal_component,
            },
        ),
        matched_skills=matched_required,
        missing_skills=missing_required,
        recommendation=_recommendation_for(jd_match_score, resume_quality_score),
        warnings=warnings,
        summary=_build_summary(jd_match_score, resume_quality_score, matched_required, missing_requi
    )

def score_resume_quality(
    *,
    resume_id: str,
    normalized_resume,
    resume_text: str,
) -> ResumeQualityResult:
    request = ScoreResumeRequest(
        resume_id=resume_id,
        normalized_resume=normalized_resume,
        resume_text=resume_text,
        job_context=JobContext(),
    )
```

```
    completeness_component = _score_completeness(request)
    structure_component = _score_structure(request)
    contact_component = _score_contactability(request)
    signal_component = _score_signal_quality(request)
    score = int(round(completeness_component + structure_component + contact_component + signal_compo

    return ResumeQualityResult(
        score=score,
        breakdown={
            "completeness": completeness_component,
            "structure": structure_component,
            "contactability": contact_component,
            "signal_quality": signal_component,
        },
        recommendation=_quality_recommendation_for(score),
        summary=_quality_summary_for(
            score=score,
            normalized_resume=request.normalized_resume,
        ),
    )

def _canonicalize_skills(skills: list[str]) -> list[str]:
    canonicalized: list[str] = []
    for skill in skills:
        lowered = skill.lower().strip()
        matched = False
        for canonical, aliases in SKILL_ALIASES.items():
            if lowered == canonical.lower() or lowered in aliases:
                canonicalized.append(canonical)
                matched = True
                break
        if not matched and skill.strip():
            canonicalized.append(skill.strip())
    return sorted(OrderedDict.fromkeys(canonicalized))

def _job_keywords(job_context: JobContext) -> list[str]:
    keywords = list(job_context.keywords)
    if job_context.title:
        keywords.extend(re.findall(r"[A-Za-z][A-Za-z.+#/-]{2,}", job_context.title))
    if job_context.description:
        keywords.extend(re.findall(r"[A-Za-z][A-Za-z.+#/-]{3,}", job_context.description))
    return sorted(OrderedDict.fromkeys(keyword.lower() for keyword in keywords))

def _keyword_overlap_ratio(keywords: list[str], resume_text: str, current_role: str | None) -> float
    if not keywords:
        return 1.0
    searchable = f"{resume_text}\n{current_role or ''}".lower()
    matched = sum(1 for keyword in keywords if keyword in searchable)
    return matched / len(keywords)

def _experience_alignment(experience_years: float, job_context: JobContext) -> float:
    minimum = job_context.experience_min_years
    maximum = job_context.experience_max_years
    if minimum is None and maximum is None:
        return 1.0 if experience_years > 0 else 0.5
    if minimum is None:
        return 1.0 if experience_years <= maximum else max(0.7, maximum / max(experience_years, 1))
    if experience_years >= minimum:
        if maximum is None or experience_years <= maximum:
            return 1.0
        return max(0.75, maximum / max(experience_years, 1))
    gap = minimum - experience_years
    return max(0.2, 1 - (gap / max(minimum, 1)))

def _score_completeness(request: ScoreResumeRequest) -> float:
    resume = request.normalized_resume
    checks = (
        bool(resume.full_name),
        bool(resume.emails),
        bool(resume.phones),
        bool(resume.current_location),
        bool(resume.skills),
        bool(resume.education),
        bool(resume.current_role),
        bool(resume.experience_summary),
        bool(resume.projects),
        bool(resume.certifications),
```

```
    )
    return round(
        sum(1 for check in checks if check) / len(checks) * SETTINGS.quality_score_weights.completen
        2,
    )

def _score_structure(request: ScoreResumeRequest) -> float:
    resume = request.normalized_resume
    ratio = 0.0
    if resume.experience_summary:
        ratio += 0.25
    if resume.skills:
        ratio += 0.2
    if resume.education:
        ratio += 0.2
    if resume.employment_history or resume.current_role:
        ratio += 0.2
    if resume.projects or resume.certifications:
        ratio += 0.15
    return round(ratio * SETTINGS.quality_score_weights.structure, 2)

def _score_contactability(request: ScoreResumeRequest) -> float:
    resume = request.normalized_resume
    score = 0.0
    if resume.emails:
        score += 8
    if resume.phones:
        score += 8
    if resume.links:
        score += 2
    if resume.current_location:
        score += 2
    return round(min(score, SETTINGS.quality_score_weights.contactability), 2)

def _score_signal_quality(request: ScoreResumeRequest) -> float:
    score = 0.0
    text_length = len(request.resume_text.strip())
    if text_length >= 300:
        score += 6
    elif text_length >= 160:
        score += 4
    if len(request.normalized_resume.skills) >= 5:
        score += 4
    elif request.normalized_resume.skills:
        score += 2
    if request.normalized_resume.employment_history or request.normalized_resume.education:
        score += 3
    if request.normalized_resume.projects:
        score += 2
    if request.normalized_resume.certifications:
        score += 2
    if request.normalized_resume.notice_period or request.normalized_resume.current_ctc or request.n
        score += 2
    return round(min(score, SETTINGS.quality_score_weights.signal_quality), 2)

def _recommendation_for(jd_match_score: int, quality_score: int) -> str:
    if jd_match_score >= 80 and quality_score >= 55:
        return "SHORTLIST"
    if jd_match_score >= 60 or quality_score >= 60:
        return "REVIEW"
    return "REJECT"

def _quality_recommendation_for(score: int) -> str:
    if score >= 75:
        return "STRONG"
    if score >= 55:
        return "REVIEW"
    return "IMPROVE"

def _quality_summary_for(*, score: int, normalized_resume) -> str:
    strengths: list[str] = []
    gaps: list[str] = []

    if normalized_resume.skills:
        strengths.append(f"{len(normalized_resume.skills)} skills extracted")
    if normalized_resume.education:
```

```
        strengths.append("education parsed")
    if normalized_resume.emails and normalized_resume.phones:
        strengths.append("contact details complete")

    if not normalized_resume.current_location:
        gaps.append("location missing")
    if not normalized_resume.employment_history and not normalized_resume.current_role:
        gaps.append("role history weak")
    if not normalized_resume.links:
        gaps.append("links missing")

    strength_text = ", ".join(strengths[:3]) if strengths else "basic profile extracted"
    if gaps:
        return f"Resume quality {score}/100. Strengths: {strength_text}. Gaps: {', '.join(gaps[:3])}
    return f"Resume quality {score}/100. Strengths: {strength_text}."

def _build_summary(jd_match_score: int, quality_score: int, matched_skills: list[str], missing_skills
    if matched_skills and missing_skills:
        return (
            f"JD match {jd_match_score}/100 and resume quality {quality_score}/100. "
            f"Strongest matches: {', '.join(matched_skills[:3])}. Missing: {', '.join(missing_skills
        )
    if matched_skills:
        return (
            f"JD match {jd_match_score}/100 and resume quality {quality_score}/100. "
            f"Strong alignment on {', '.join(matched_skills[:3])}."
        )
    return f"JD match {jd_match_score}/100 and resume quality {quality_score}/100. Resume needs close

def _weighted_experience_years(request: ScoreResumeRequest) -> float:
    if request.normalized_resume.total_relevant_months:
        weighted_months = request.normalized_resume.full_time_months + (request.normalized_resume.int
        return weighted_months / 12
    return estimate_total_experience_months(request.normalized_resume.employment_history) / 12

def _skill_evidence_strength(evidence_records: list[SkillEvidenceRecord]) -> dict[str, float]:
    strength_map: dict[str, float] = {}
    weights = {
        "primary": 1.0,
        "project": 0.9,
        "certification": 0.85,
        "mention": 0.7,
    }
    for evidence in evidence_records:
        strength_map[evidence.skill] = max(strength_map.get(evidence.skill, 0.0), weights.get(evidenc
    return strength_map
```
▼ Show more
ReadFile

parser.py
```
from __future__ import annotations

import hashlib
import re

from .extractors import ExtractionError, UnsupportedResumeError, extract_resume
from .normalization import normalize_resume
from .schemas import CandidateType, ParseResumeResponse, ParserMetadata
from .scoring import score_resume_quality
from .settings import SETTINGS

def parse_resume_document(
    *,
    file_bytes: bytes,
    filename: str,
    resume_id: str,
    candidate_type: CandidateType,
    content_type: str | None = None,
) -> ParseResumeResponse:
    if len(file_bytes) > SETTINGS.max_file_size_bytes:
        raise ExtractionError(f"Resume exceeds the {SETTINGS.max_file_size_bytes // (1024 * 1024)} MB

    try:
        extraction = extract_resume(file_bytes, filename=filename, content_type=content_type)
    except ExtractionError as exc:
        if "does not contain extractable text" in str(exc).lower():
```

```
            raise UnsupportedResumeError("Image-based or scanned resumes are not supported in v1.") f
        raise
    if len(extraction.text.strip()) < SETTINGS.minimum_text_characters:
        raise UnsupportedResumeError("Image-based or scanned resumes are not supported in v1.")

    normalized, warnings, missing_fields, section_order, sensitive_findings = normalize_resume(
        extraction.lines,
        extraction.text,
        candidate_type,
    )
    fingerprint = _content_fingerprint(extraction.text)

    metadata = ParserMetadata(
        filename=filename,
        file_type=extraction.file_type,
        content_type=content_type or extraction.content_type,
        file_size_bytes=len(file_bytes),
        line_count=len(extraction.lines),
        page_count=int(extraction.metadata.get("page_count", 1) or 1),
        extractor_name=str(extraction.metadata.get("extractor_name") or extraction.file_type),
        section_order=section_order,
        detected_columns=extraction.detected_columns,
        is_scanned=False,
        content_fingerprint=fingerprint,
        sensitive_findings=sensitive_findings,
        extracted_text=extraction.text,
    )

    if extraction.detected_columns > 1:
        warnings.append("Two-column layout detected; section ordering was normalized before extracti

    resume_quality = score_resume_quality(
        resume_id=resume_id,
        normalized_resume=normalized,
        resume_text=extraction.text,
    )

    return ParseResumeResponse(
        resume_id=resume_id,
        candidate_type=candidate_type,
        normalized_resume=normalized,
        warnings=warnings,
        missing_fields=missing_fields,
        resume_quality=resume_quality,
        parser_metadata=metadata,
    )

def _content_fingerprint(text: str) -> str:
    normalized = re.sub(r"\s+", " ", text).strip().lower()
    return hashlib.sha256(normalized.encode("utf-8")).hexdigest()
▼ Show more
ReadFile

constants.py
from __future__ import annotations

SECTION_ALIASES: dict[str, tuple[str, ...]] = {
    "summary": (
        "summary",
        "professional summary",
        "profile summary",
        "career summary",
        "objective",
        "career objective",
        "profile",
        "about me",
    ),
    "skills": (
        "skills",
        "technical skills",
        "core skills",
        "key skills",
        "competencies",
        "technology stack",
        "tools and technologies",
    ),
```

```
    "experience": (
        "experience",
        "work experience",
        "professional experience",
        "employment history",
        "career history",
    ),
    "internships": (
        "internships",
        "internship",
        "industrial training",
        "training",
        "apprenticeship",
    ),
    "projects": (
        "projects",
        "academic projects",
        "personal projects",
        "key projects",
    ),
    "education": (
        "education",
        "educational qualifications",
        "academic details",
        "academics",
        "qualification",
    ),
    "certifications": (
        "certifications",
        "courses",
        "certificates",
        "licenses",
    ),
    "achievements": (
        "achievements",
        "accomplishments",
        "awards",
        "honors",
    ),
}

DEGREE_ALIASES: dict[str, tuple[str, ...]] = {
    "B.Tech": ("b tech", "b.tech", "b-tech", "b.tech.", "bachelor of technology"),
    "B.E": ("b e", "b.e", "bachelor of engineering"),
    "M.Tech": ("m tech", "m.tech", "master of technology"),
    "M.E": ("m e", "m.e", "master of engineering"),
    "MCA": ("mca", "master of computer applications"),
    "BCA": ("bca", "bachelor of computer applications"),
    "B.Sc": ("bsc", "b.sc", "bachelor of science"),
    "M.Sc": ("msc", "m.sc", "master of science"),
    "B.Com": ("bcom", "b.com", "bachelor of commerce"),
    "MBA": ("mba", "master of business administration"),
    "PGDM": ("pgdm", "post graduate diploma in management"),
    "Diploma": ("diploma", "polytechnic"),
    "Intermediate": ("intermediate", "mpc", "12th", "class 12"),
    "SSC": ("ssc", "class 10", "10th"),
}

SKILL_ALIASES: dict[str, tuple[str, ...]] = {
    "Python": ("python",),
    "C": ("c",),
    "C++": ("c++", "cpp"),
    "C#": ("c#", "dotnet", ".net"),
    "Java": ("java",),
    "JavaScript": ("javascript", "js"),
    "TypeScript": ("typescript", "ts"),
    "Node.js": ("node", "nodejs", "node.js"),
    "React": ("react", "reactjs", "react.js"),
    "Angular": ("angular", "angularjs"),
    "Vue.js": ("vue", "vuejs", "vue.js"),
    "Spring Boot": ("spring boot", "springboot"),
    "Spring": ("spring framework", "spring"),
    "Django": ("django",),
    "FastAPI": ("fastapi",),
    "Flask": ("flask",),
    "REST APIs": ("rest api", "rest apis", "restful api", "restful apis"),
    "GraphQL": ("graphql",),
```

```
    "SQL": ("sql",),
    "Oracle": ("oracle", "oracle db"),
    "MySQL": ("mysql",),
    "PostgreSQL": ("postgresql", "postgres"),
    "MongoDB": ("mongodb", "mongo db"),
    "Redis": ("redis",),
    "AWS": ("aws", "amazon web services"),
    "Azure": ("azure",),
    "GCP": ("gcp", "google cloud", "google cloud platform"),
    "Docker": ("docker",),
    "Kubernetes": ("kubernetes", "k8s"),
    "Terraform": ("terraform",),
    "Jenkins": ("jenkins",),
    "GitHub Actions": ("github actions",),
    "CI/CD": ("ci/cd", "ci cd", "continuous integration", "continuous delivery"),
    "Git": ("git", "github", "gitlab", "bitbucket"),
    "Data Structures": ("data structures", "dsa"),
    "Algorithms": ("algorithms",),
    "HTML": ("html", "html5"),
    "CSS": ("css", "css3"),
    "Bootstrap": ("bootstrap",),
    "Tailwind CSS": ("tailwind", "tailwind css"),
    "Linux": ("linux",),
    "Flutter": ("flutter",),
    "Android": ("android",),
    "Raspberry Pi": ("raspberry pi",),
    "Arduino": ("arduino",),
    "IoT": ("iot", "internet of things"),
    "Pandas": ("pandas",),
    "NumPy": ("numpy",),
    "Machine Learning": ("machine learning", "ml"),
    "Deep Learning": ("deep learning", "dl"),
    "NLP": ("nlp", "natural language processing"),
    "Excel": ("excel", "ms excel", "microsoft excel"),
}

INDIAN_LOCATIONS: dict[str, str] = {
    "ahmedabad": "Ahmedabad, Gujarat",
    "bengaluru": "Bengaluru, Karnataka",
    "bangalore": "Bengaluru, Karnataka",
    "chandigarh": "Chandigarh",
    "chennai": "Chennai, Tamil Nadu",
    "coimbatore": "Coimbatore, Tamil Nadu",
    "delhi": "Delhi",
    "gurgaon": "Gurugram, Haryana",
    "gurugram": "Gurugram, Haryana",
    "hyderabad": "Hyderabad, Telangana",
    "indore": "Indore, Madhya Pradesh",
    "jaipur": "Jaipur, Rajasthan",
    "kochi": "Kochi, Kerala",
    "kolkata": "Kolkata, West Bengal",
    "lucknow": "Lucknow, Uttar Pradesh",
    "mumbai": "Mumbai, Maharashtra",
    "nagpur": "Nagpur, Maharashtra",
    "noida": "Noida, Uttar Pradesh",
    "pune": "Pune, Maharashtra",
    "trivandrum": "Thiruvananthapuram, Kerala",
    "thiruvananthapuram": "Thiruvananthapuram, Kerala",
    "vadodara": "Vadodara, Gujarat",
    "vijayawada": "Vijayawada, Andhra Pradesh",
    "visakhapatnam": "Visakhapatnam, Andhra Pradesh",
}

SENSITIVE_PATTERNS: dict[str, str] = {
    "aadhaar": r"\b(?:aadhaar|aadhar)\b|\b\d{4}\s?\d{4}\s?\d{4}\b",
    "pan": r"(?:\bpan\b[^\n]{0,24}\b[A-Z]{5}\d{4}[A-Z]\b)|(?:\b[A-Z]{5}\d{4}[A-Z]\b[^\n]{0,12}\bpan\b
    "passport": r"\bpassport\b|\b[A-PR-WYa-pr-wy][1-9]\d{6}\b",
    "family_details": r"\b(father'?s name|mother'?s name|spouse|husband|wife|children)\b",
    "religion": r"\breligion\b",
    "caste": r"\bcaste\b",
    "marital_status": r"\bmarital status\b",
    "date_of_birth": r"\b(date of birth|dob)\b",
}

NOTICE_PERIOD_PATTERNS: tuple[str, ...] = (
    r"\bnotice period\b\s*[:\-]?\s*([A-Za-z0-9 +]+)",
    r"\bnp\b\s*[:\-]?\s*([A-Za-z0-9 +]+)",
```

```
)

CTC_PATTERNS: dict[str, str] = {
    "current_ctc": r"\b(current ctc|current compensation|current salary)\b\s*[:\-]?\s*([^\n,;]+)",
    "expected_ctc": r"\b(expected ctc|expected compensation|expected salary)\b\s*[:\-]?\s*([^\n,;]+)"
}

WORK_AUTHORIZATION_PATTERNS: tuple[str, ...] = (
    r"\b(?:work authorization|authorized to work|work permit|visa status)\b\s*[:\-]?\s*([^\n,;]+)",
    r"\b(?:us visa|h1b|citizen|permanent resident)\b\s*[:\-]?\s*([^\n,;]+)",
)

PREFERRED_LOCATION_PATTERNS: tuple[str, ...] = (
    r"\bpreferred location\b\s*[:\-]?\s*([^\n,;]+)",
    r"\bopen to relocate(?:ion)?\b\s*[:\-]?\s*([^\n,;]+)",
)

MONTH_NAMES: tuple[str, ...] = (
    "jan",
    "january",
    "feb",
    "february",
    "mar",
    "march",
    "apr",
    "april",
    "may",
    "jun",
    "june",
    "jul",
    "july",
    "aug",
    "august",
    "sep",
    "sept",
    "september",
    "oct",
    "october",
    "nov",
    "november",
    "dec",
    "december",
)
▼ Show more
ReadFile

settings.py
from __future__ import annotations

from dataclasses import dataclass, field

@dataclass(frozen=True)
class ScoreWeights:
    skills: int = 50
    experience: int = 30
    keywords: int = 20

@dataclass(frozen=True)
class ResumeQualityWeights:
    completeness: int = 40
    structure: int = 25
    contactability: int = 20
    signal_quality: int = 15

@dataclass(frozen=True)
class AppSettings:
    max_file_size_bytes: int = 5 * 1024 * 1024
    supported_extensions: tuple[str, ...] = (".pdf", ".docx")
    blocked_extensions: tuple[str, ...] = (".doc",)
    minimum_text_characters: int = 60
    rate_limit_requests_per_window: int = 30
    rate_limit_window_seconds: int = 60
    jd_score_weights: ScoreWeights = field(default_factory=ScoreWeights)
    quality_score_weights: ResumeQualityWeights = field(default_factory=ResumeQualityWeights)

SETTINGS = AppSettings()
```

▼ Show more
ReadFile

normalization.py

```
from __future__ import annotations

import re
from collections import OrderedDict, defaultdict
from datetime import date

from .constants import (
    CTC_PATTERNS,
    DEGREE_ALIASES,
    INDIAN_LOCATIONS,
    MONTH_NAMES,
    NOTICE_PERIOD_PATTERNS,
    PREFERRED_LOCATION_PATTERNS,
    SECTION_ALIASES,
    SENSITIVE_PATTERNS,
    SKILL_ALIASES,
    WORK_AUTHORIZATION_PATTERNS,
)
from .schemas import (
    CandidateType,
    CertificationRecord,
    EducationRecord,
    EmploymentRecord,
    NormalizedResume,
    ProjectRecord,
    SkillEvidenceRecord,
)

EMAIL_RE = re.compile(r"\b[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[A-Za-z]{2,}\b")
PHONE_RE = re.compile(r"(?:\+?91[\s\-]?)?(?:\(?0?\)?[\s\-]?)?[6-9]\d[\d\s\-]{8,12}")
URL_RE = re.compile(r"\b(?:https?://|www\.)[^\s|,;]+", re.I)
DATE_RANGE_RE = re.compile(
    rf"((?:{'|'.join(MONTH_NAMES)})\s+\d{{4}}|\d{{4}})\s*(?:-|to|–|—)\s*((?:{'|'.join(MONTH_NAMES)})
    re.I,
)
EXPERIENCE_YEARS_RE = re.compile(r"(\d+(?:\.\d+)?)\+?\s+(?:years?|yrs?)", re.I)
YEAR_RE = re.compile(r"\b(20\d{2}|19\d{2})\b")
CGPA_RE = re.compile(r"\b(?:cgpa|gpa)\b\s*[:\-]?\s*(\d+(?:\.\d+)?)", re.I)
PERCENT_RE = re.compile(r"(\d{2}(?:\.\d+)?)\s*%")
ROLE_COMPANY_RE = re.compile(
    r"^(?P<title>[A-Za-z0-9 /&().+-]+?)\s*(?:\||@| at )\s*(?P<company>[A-Za-z0-9 /&().,+-]+?)(?:\s*\
    re.I,
)
PROJECT_HEADER_RE = re.compile(r"^(?P<title>[A-Za-z][A-Za-z0-9 /&().+-]{2,})(?:\s*\((?P<stack>[^)]+)
CERT_DATE_RE = re.compile(r"\b(?:jan|feb|mar|apr|may|jun|jul|aug|sep|sept|oct|nov|dec)[a-z]*\s+\d{4}

def normalize_resume(lines: list[str], raw_text: str, candidate_type: CandidateType) -> tuple[Normali
    cleaned_lines = _clean_lines(lines)
    section_map = _split_sections(cleaned_lines)
    header_lines = section_map.get("header", cleaned_lines[:5])
    sensitive_findings = _detect_sensitive_findings(raw_text)
    warnings: list[str] = []
    if sensitive_findings:
        warnings.append("Sensitive personal identifiers were detected and excluded from the normaliz

    emails = sorted(OrderedDict.fromkeys(match.lower() for match in EMAIL_RE.findall(raw_text)))
    phones = sorted(OrderedDict.fromkeys(filter(None, (_normalize_phone(match) for match in PHONE_RE
    links = sorted(OrderedDict.fromkeys(_normalize_link(match) for match in URL_RE.findall(raw_text)
    full_name = _extract_name(header_lines, cleaned_lines)
    current_location = _extract_location(header_lines, cleaned_lines, raw_text)
    projects = _extract_projects(section_map.get("projects", []))
    certifications = _extract_certifications(section_map.get("certifications", []))
    skills, skill_evidence = _extract_skills(raw_text, section_map, projects, certifications)
    education = _extract_education(section_map.get("education", cleaned_lines))
    employment_history = _extract_employment_history(section_map, candidate_type)
    full_time_months, internship_months, total_relevant_months = calculate_experience_totals(employme
    current_role = _extract_current_role(header_lines, cleaned_lines, employment_history)
    experience_summary = _extract_experience_summary(section_map, raw_text, total_relevant_months)
    notice_period = _extract_pattern_value(raw_text, NOTICE_PERIOD_PATTERNS)
    current_ctc = _extract_named_value(raw_text, CTC_PATTERNS["current_ctc"])
    expected_ctc = _extract_named_value(raw_text, CTC_PATTERNS["expected_ctc"])
    preferred_location = _extract_pattern_value(raw_text, PREFERRED_LOCATION_PATTERNS)
    work_authorization = _extract_pattern_value(raw_text, WORK_AUTHORIZATION_PATTERNS)
```

```
    normalized = NormalizedResume(
        full_name=full_name,
        emails=emails,
        phones=phones,
        current_location=current_location,
        links=links,
        skills=skills,
        skill_evidence=skill_evidence,
        experience_summary=experience_summary,
        employment_history=employment_history,
        education=education,
        projects=projects,
        certifications=certifications,
        current_role=current_role,
        full_time_months=full_time_months,
        internship_months=internship_months,
        total_relevant_months=total_relevant_months,
        notice_period=notice_period,
        current_ctc=current_ctc,
        expected_ctc=expected_ctc,
        preferred_location=preferred_location,
        work_authorization=work_authorization,
    )

    missing_fields = [
        field_name
        for field_name in (
            "full_name",
            "emails",
            "phones",
            "current_location",
            "skills",
            "education",
            "current_role",
        )
        if _is_missing(getattr(normalized, field_name))
    ]

    return normalized, warnings, missing_fields, list(section_map.keys()), sensitive_findings

def estimate_total_experience_months(employment_history: list[EmploymentRecord]) -> int:
    full_time_months, internship_months, _ = calculate_experience_totals(employment_history)
    return int(round(full_time_months + (internship_months * 0.75)))

def calculate_experience_totals(employment_history: list[EmploymentRecord]) -> tuple[int, int, int]:
    full_time_months = 0
    internship_months = 0
    for record in employment_history:
        months = _months_for_record(record)
        if months <= 0:
            continue
        if record.employment_type == "internship":
            internship_months += months
        else:
            full_time_months += months
    return full_time_months, internship_months, full_time_months + internship_months

def _clean_lines(lines: list[str]) -> list[str]:
    cleaned: list[str] = []
    for raw_line in lines:
        line = raw_line.replace("\u200b", " ").replace("\ufeff", " ")
        line = re.sub(r"[•●■◆■]", " ", line)
        line = re.sub(r"\s+", " ", line).strip(" -:\t")
        if line:
            cleaned.append(line)
    return cleaned

def _split_sections(lines: list[str]) -> dict[str, list[str]]:
    sections: dict[str, list[str]] = OrderedDict()
    current_section = "header"
    sections[current_section] = []

    for line in lines:
        section_key = _match_section_heading(line)
        if section_key:
```

```
            current_section = section_key
            sections.setdefault(current_section, [])
            continue
        sections.setdefault(current_section, []).append(line)

    return sections

def _match_section_heading(line: str) -> str | None:
    normalized = re.sub(r"[^a-z0-9 ]", "", line.lower()).strip()
    for section, aliases in SECTION_ALIASES.items():
        if normalized in aliases:
            return section
    return None

def _detect_sensitive_findings(raw_text: str) -> list[str]:
    findings: list[str] = []
    for label, pattern in SENSITIVE_PATTERNS.items():
        if re.search(pattern, raw_text, re.I):
            findings.append(label)
    return findings

def _extract_name(header_lines: list[str], cleaned_lines: list[str]) -> str | None:
    candidates: list[tuple[int, str]] = []
    for line in header_lines + cleaned_lines[:6]:
        if _looks_like_contact_line(line) or _match_section_heading(line):
            continue
        if len(line) > 48 or not (2 <= len(line.split()) <= 5):
            continue
        if "|" in line or "," in line:
            continue
        score = 0
        if line.istitle():
            score += 2
        if line.isupper():
            score += 1
        if header_lines and line in header_lines[:2]:
            score += 2
        candidates.append((score, line))

    if not candidates:
        return None

    best = sorted(candidates, key=lambda item: (-item[0], len(item[1])))[0][1]
    return " ".join(word.capitalize() for word in best.split())

def _extract_location(header_lines: list[str], cleaned_lines: list[str] | str, raw_text: str | None =
    if raw_text is None and isinstance(cleaned_lines, str):
        raw_text = cleaned_lines
        cleaned_lines = header_lines
    assert raw_text is not None
    for line in header_lines[:4]:
        if _looks_like_contact_line(line) or "http" in line.lower() or "www." in line.lower():
            continue
        header_part = line.split("|", 1)[0].strip()
        if "," in header_part and not EMAIL_RE.search(header_part) and not PHONE_RE.search(header_par
            return header_part
    search_window = "\n".join(cleaned_lines[:20]) + "\n" + raw_text[:1000]
    for alias, normalized in INDIAN_LOCATIONS.items():
        if re.search(rf"\b{re.escape(alias)}\b", search_window, re.I):
            return normalized
    explicit_match = re.search(r"\b(?:location|current location)\b\s*[:\-]?\s*([^\n|,;]+)", raw_text,
    return explicit_match.group(1).strip() if explicit_match else None

def _extract_skills(
    raw_text: str,
    section_map: dict[str, list[str]] | list[str],
    projects: list[ProjectRecord] | None = None,
    certifications: list[CertificationRecord] | None = None,
) -> tuple[list[str], list[SkillEvidenceRecord]]:
    if isinstance(section_map, list):
        section_map = {"skills": section_map}
    projects = projects or []
    certifications = certifications or []
    evidence_map: dict[str, set[tuple[str, str]]] = defaultdict(set)
    contexts = (
        ("primary", "skills", "\n".join(section_map.get("skills", []))),
```

```
            (
                "project",
                "projects",
                "\n".join(
                    " ".join(part for part in [project.title, project.summary or "", ", ".join(project.te
                    for project in projects
                ),
            ),
            (
                "certification",
                "certifications",
                "\n".join(
                    " ".join(part for part in [cert.name, cert.issuer or "", cert.issued_date or ""] if p
                    for cert in certifications
                ),
            ),
            ("mention", "summary", raw_text),
        )

        for evidence_type, source_section, text in contexts:
            if not text.strip():
                continue
            for canonical in _match_skills(text):
                evidence_map[canonical].add((evidence_type, source_section))

        skills = sorted(evidence_map.keys())
        evidence: list[SkillEvidenceRecord] = []
        priority = {"primary": 0, "project": 1, "certification": 2, "mention": 3}
        for skill in skills:
            for evidence_type, source_section in sorted(evidence_map[skill], key=lambda item: priority.ge
                evidence.append(
                    SkillEvidenceRecord(
                        skill=skill,
                        evidence_type=evidence_type,
                        source_section=source_section,
                    )
                )
        return skills, evidence

    def _match_skills(text: str) -> list[str]:
        normalized_text = text.lower()
        found: list[str] = []
        consumed_spans: list[tuple[int, int]] = []
        alias_entries = sorted(
            ((canonical, alias) for canonical, aliases in SKILL_ALIASES.items() for alias in aliases),
            key=lambda item: len(item[1]),
            reverse=True,
        )
        for canonical, alias in alias_entries:
            if canonical in found:
                continue
            pattern = re.compile(rf"(?<![A-Za-z]){re.escape(alias)}(?![A-Za-z])", re.I)
            alias_spans = [
                match.span()
                for match in pattern.finditer(normalized_text)
                if not _spans_overlap(match.span(), consumed_spans)
            ]
            if not alias_spans:
                continue
            found.append(canonical)
            consumed_spans.extend(alias_spans)
        return sorted(OrderedDict.fromkeys(found))

    def _spans_overlap(candidate: tuple[int, int], existing: list[tuple[int, int]]) -> bool:
        start, end = candidate
        return any(start < other_end and end > other_start for other_start, other_end in existing)

    def _extract_education(lines: list[str]) -> list[EducationRecord]:
        entries: list[EducationRecord] = []
        for line in lines:
            normalized_line = line.lower()
            degree = None
            for canonical, aliases in DEGREE_ALIASES.items():
                if any(alias in normalized_line for alias in aliases):
                    degree = canonical
                    break
            if not degree:
```

```
            continue
        year_matches = YEAR_RE.findall(line)
        cgpa_match = CGPA_RE.search(line)
        percent_match = PERCENT_RE.search(line)
        entries.append(
            EducationRecord(
                degree=degree,
                specialization=_extract_specialization(line, degree),
                institution=_extract_institution(line, degree),
                completion_year=int(year_matches[-1]) if year_matches else None,
                score=cgpa_match.group(1) if cgpa_match else (percent_match.group(1) + "%" if percent
            )
        )

    deduped: list[EducationRecord] = []
    seen: set[tuple[str | None, str | None, int | None]] = set()
    for entry in entries:
        key = (entry.degree, entry.institution, entry.completion_year)
        if key in seen:
            continue
        seen.add(key)
        deduped.append(entry)
    return deduped

def _extract_specialization(line: str, degree: str) -> str | None:
    parts = [part.strip() for part in re.split(r"[-|,]", line) if part.strip()]
    for part in parts:
        lowered = part.lower()
        if degree.lower().replace(".", "") in re.sub(r"[^a-z]", "", lowered):
            continue
        if any(token in lowered for token in ("computer", "information", "electronics", "mechanical",
            return part
    return None

def _extract_institution(line: str, degree: str) -> str | None:
    if " - " in line:
        prefix, remainder = line.split(" - ", 1)
        if degree.lower().replace(".", "") in re.sub(r"[^a-z]", "", prefix.lower()):
            cleaned = re.sub(r"\(?\b(?:19|20)\d{2}\s*[-–]\s*(?:19|20)\d{2}\b\)?", "", remainder)
            cleaned = re.sub(r"\(?\b(?:19|20)\d{2}\b\)?", "", cleaned)
            cleaned = cleaned.strip(" -()")
            if cleaned:
                return cleaned
    parts = [part.strip() for part in re.split(r"[-|,]", line) if part.strip()]
    for part in parts:
        lowered = part.lower()
        if degree.lower() in lowered:
            continue
        if any(word in lowered for word in ("university", "college", "institute", "school")):
            return part
    return None

def _extract_projects(lines: list[str]) -> list[ProjectRecord]:
    projects: list[ProjectRecord] = []
    active: dict[str, object] | None = None
    for line in lines:
        if _looks_like_project_header(line):
            if active:
                projects.append(_project_from_active(active))
            active = _seed_project(line)
            continue
        if active is not None:
            summary_lines = active.setdefault("summary_lines", [])
            summary_lines.append(line)
    if active:
        projects.append(_project_from_active(active))
    return projects

def _looks_like_project_header(line: str) -> bool:
    if len(line) < 4 or len(line) > 120:
        return False
    if line.endswith(":") or line.lower().startswith(("developed", "built", "implemented", "enabled",
        return False
    return bool(PROJECT_HEADER_RE.match(line))

def _seed_project(line: str) -> dict[str, object]:
    match = PROJECT_HEADER_RE.match(line)
```

```
    title = line
    tech_stack: list[str] = []
    if match:
        title = match.group("title").strip()
        inline_stack = match.group("stack") or match.group("pipe")
        if inline_stack:
            tech_stack = _match_skills(inline_stack)
    return {"title": title, "tech_stack": tech_stack, "summary_lines": []}

def _project_from_active(active: dict[str, object]) -> ProjectRecord:
    summary = " ".join(line for line in active.get("summary_lines", []) if isinstance(line, str)).str
    tech_stack = list(active.get("tech_stack", []))
    if summary:
        tech_stack = sorted(OrderedDict.fromkeys(tech_stack + _match_skills(summary)))
    return ProjectRecord(
        title=str(active["title"]).strip(),
        tech_stack=tech_stack,
        summary=summary,
        source_section="projects",
    )

def _extract_certifications(lines: list[str]) -> list[CertificationRecord]:
    certifications: list[CertificationRecord] = []
    for line in lines:
        cleaned = line.strip(" -")
        if len(cleaned) < 4:
            continue
        issued_date_match = CERT_DATE_RE.search(cleaned)
        issued_date = issued_date_match.group(0) if issued_date_match else None
        working = cleaned
        if issued_date:
            working = working.replace(issued_date, "").strip(" -()")
        if " - " in working:
            left, right = working.split(" - ", 1)
            if len(left.split()) >= 2:
                certifications.append(CertificationRecord(name=left.strip(), issuer=right.strip() or
                continue
        parts = [part.strip() for part in re.split(r"[–|]", working) if part.strip()]
        if len(parts) >= 2:
            certifications.append(CertificationRecord(name=parts[0], issuer=parts[1], issued_date=iss
            continue
        certifications.append(CertificationRecord(name=working.strip(), issuer=None, issued_date=issu
    return certifications

def _extract_employment_history(section_map: dict[str, list[str]], candidate_type: CandidateType) ->
    history: list[EmploymentRecord] = []
    primary_sections = ["experience", "internships"] if candidate_type == CandidateType.fresher else
    for section_name in primary_sections:
        history.extend(_parse_experience_entries(section_map.get(section_name, []), section_name.rstr
    return history

def _parse_experience_entries(lines: list[str], employment_type: str) -> list[EmploymentRecord]:
    entries: list[EmploymentRecord] = []
    active: dict[str, str | None] | None = None

    for line in lines:
        if "|" in line and DATE_RANGE_RE.search(line):
            if active:
                entries.append(_employment_from_active(active, employment_type))
            parts = [part.strip() for part in line.split("|") if part.strip()]
            title = parts[0] if parts else None
            company = parts[1] if len(parts) > 1 else None
            dates = parts[2] if len(parts) > 2 else line
            location = parts[3] if len(parts) > 3 else None
            active = {
                "title": title,
                "company": company,
                "dates": dates,
                "description": "",
                "location": location,
            }
            continue

        role_company_match = ROLE_COMPANY_RE.match(line)
        date_range_match = DATE_RANGE_RE.search(line)
        if role_company_match or date_range_match:
            if active:
```

```
                entries.append(_employment_from_active(active, employment_type))
            active = {
                "title": role_company_match.group("title").strip() if role_company_match else None,
                "company": role_company_match.group("company").strip() if role_company_match else No
                "dates": (role_company_match.group("dates") if role_company_match else line).strip()
                "description": "",
                "location": None,
            }
            continue
        if active is not None:
            active["description"] = f"{active.get('description', '')} {line}".strip()

    if active:
        entries.append(_employment_from_active(active, employment_type))
    return entries

def _employment_from_active(active: dict[str, str | None], employment_type: str) -> EmploymentRecord
    dates_text = active.get("dates") or ""
    date_match = DATE_RANGE_RE.search(dates_text)
    start_date = date_match.group(1).title() if date_match else None
    end_date = date_match.group(2).title() if date_match else None
    return EmploymentRecord(
        company=(active.get("company") or "").strip() or None,
        title=(active.get("title") or "").strip() or None,
        start_date=start_date,
        end_date=end_date,
        employment_type=employment_type,
        location=(active.get("location") or "").strip() or None,
        description=(active.get("description") or "").strip() or None,
        is_current=bool(end_date and end_date.lower() in {"present", "current"}),
    )

def _extract_current_role(header_lines: list[str], cleaned_lines: list[str], employment_history: lis
    for record in employment_history:
        if record.is_current and record.title:
            return record.title
    for record in employment_history:
        if record.title:
            return record.title
    for line in header_lines + cleaned_lines[:10]:
        undergrad_match = re.search(
            r"\b((?:[A-Za-z]+\s+){0,4}(?:undergraduate|student|intern))\b",
            line,
            re.I,
        )
        if undergrad_match:
            return undergrad_match.group(1).strip().title()
    for line in header_lines + cleaned_lines[:12]:
        if _match_section_heading(line) or _looks_like_contact_line(line):
            continue
        if len(line.split()) <= 8 and re.search(
            r"\b(engineer|developer|analyst|intern|consultant|manager|undergraduate|student)\b",
            line,
            re.I,
        ):
            return line
    return None

def _extract_experience_summary(section_map: dict[str, list[str]], raw_text: str, total_relevant_mon
    summary_lines = section_map.get("summary", [])
    if summary_lines:
        return " ".join(summary_lines[:3]).strip()
    years_match = EXPERIENCE_YEARS_RE.search(raw_text)
    if years_match:
        return f"{years_match.group(1)} years of experience"
    if total_relevant_months > 0:
        return f"{round(total_relevant_months / 12, 1)} years of relevant experience"
    return None

def _normalize_phone(raw_phone: str) -> str | None:
    digits = re.sub(r"\D", "", raw_phone)
    if len(digits) == 10 and digits[0] in "6789":
        return f"+91{digits}"
    if len(digits) == 12 and digits.startswith("91") and digits[2] in "6789":
        return f"+{digits}"
    if len(digits) == 11 and digits.startswith("0") and digits[1] in "6789":
        return f"+91{digits[1:]}"
```

```
    return None

def _normalize_link(link: str) -> str:
    if link.lower().startswith("http"):
        return link.rstrip("/").lower()
    return f"https://{link.rstrip('/')}".lower()

def _extract_pattern_value(raw_text: str, patterns: tuple[str, ...]) -> str | None:
    for pattern in patterns:
        match = re.search(pattern, raw_text, re.I)
        if match:
            return match.group(1).strip()
    return None

def _extract_named_value(raw_text: str, pattern: str) -> str | None:
    match = re.search(pattern, raw_text, re.I)
    return match.group(2).strip() if match else None

def _parse_month_year(value: str | None) -> date | None:
    if not value:
        return None
    lower = value.lower()
    year_match = YEAR_RE.search(lower)
    if not year_match:
        return None
    year = int(year_match.group(1))
    month = 1
    for index, month_name in enumerate(MONTH_NAMES[:12], start=1):
        if re.search(rf"\b{month_name}\b", lower):
            month = index
            break
    return date(year, month, 1)

def _months_for_record(record: EmploymentRecord) -> int:
    start = _parse_month_year(record.start_date)
    end = _parse_month_year(record.end_date) if record.end_date and record.end_date.lower() not in {'
    if not start or not end or end < start:
        return 0
    return max(1, (end.year - start.year) * 12 + (end.month - start.month) + 1)

def _looks_like_contact_line(line: str) -> bool:
    return bool(EMAIL_RE.search(line) or PHONE_RE.search(line) or URL_RE.search(line) or "|" in line

def _is_missing(value: object) -> bool:
    if value is None:
        return True
    if isinstance(value, str):
        return not value.strip()
    if isinstance(value, list):
        return len(value) == 0
    return False
▼ Show more
ReadFile

observability.py
from __future__ import annotations

import json
import logging
import time
from collections import defaultdict, deque
from dataclasses import dataclass, field

from .schemas import MetricsResponse
from .settings import SETTINGS

LOGGER = logging.getLogger("resume_service")
if not LOGGER.handlers:
    logging.basicConfig(level=logging.INFO)

@dataclass
class MetricsStore:
    parse_success_count: int = 0
    parse_failure_count: int = 0
    score_request_count: int = 0
    unsupported_format_reject_count: int = 0
```

```
    scanned_resume_reject_count: int = 0
    parse_latency_total_ms: float = 0.0
    score_latency_total_ms: float = 0.0

    def snapshot(self) -> MetricsResponse:
        parse_avg = self.parse_latency_total_ms / self.parse_success_count if self.parse_success_cou
        score_avg = self.score_latency_total_ms / self.score_request_count if self.score_request_cou
        return MetricsResponse(
            parse_success_count=self.parse_success_count,
            parse_failure_count=self.parse_failure_count,
            score_request_count=self.score_request_count,
            unsupported_format_reject_count=self.unsupported_format_reject_count,
            scanned_resume_reject_count=self.scanned_resume_reject_count,
            average_parse_latency_ms=round(parse_avg, 2),
            average_score_latency_ms=round(score_avg, 2),
        )

@dataclass
class RateLimiter:
    _events: dict[tuple[str, str], deque[float]] = field(default_factory=lambda: defaultdict(deque))

    def allow(self, client_id: str, route: str) -> bool:
        now = time.time()
        window = SETTINGS.rate_limit_window_seconds
        limit = SETTINGS.rate_limit_requests_per_window
        bucket = self._events[(client_id, route)]
        while bucket and bucket[0] <= now - window:
            bucket.popleft()
        if len(bucket) >= limit:
            return False
        bucket.append(now)
        return True

METRICS = MetricsStore()
RATE_LIMITER = RateLimiter()

def log_event(**payload: object) -> None:
    LOGGER.info(json.dumps(payload, ensure_ascii=True, default=str))
▼ Show more
ReadFile

multipart_parser.py
from __future__ import annotations

import re
from dataclasses import dataclass

class MultipartError(ValueError):
    """Raised when multipart parsing fails."""

@dataclass(slots=True)
class MultipartPart:
    name: str
    data: bytes
    filename: str | None = None
    content_type: str | None = None

    @property
    def text(self) -> str:
        return self.data.decode("utf-8", "ignore").strip()

_BOUNDARY_RE = re.compile(r'boundary=(?:"([^"]+)"|([^;]+))')
_DISPOSITION_PARAM_RE = re.compile(r'([a-zA-Z0-9_-]+)="([^"]*)"')

def parse_multipart(body: bytes, content_type: str | None) -> dict[str, MultipartPart]:
    if not content_type or "multipart/form-data" not in content_type.lower():
        raise MultipartError("Content-Type must be multipart/form-data")

    boundary_match = _BOUNDARY_RE.search(content_type)
    if not boundary_match:
        raise MultipartError("Multipart boundary is missing")

    boundary = (boundary_match.group(1) or boundary_match.group(2) or "").strip()
    if not boundary:
        raise MultipartError("Multipart boundary is empty")
```

```
    delimiter = f"--{boundary}".encode("utf-8")
    raw_parts = body.split(delimiter)
    parsed: dict[str, MultipartPart] = {}

    for raw_part in raw_parts[1:]:
        if raw_part.startswith(b"--"):
            break

        chunk = raw_part.strip(b"\r\n")
        if not chunk:
            continue

        header_blob, separator, payload = chunk.partition(b"\r\n\r\n")
        if not separator:
            raise MultipartError("Multipart part is missing header separator")

        headers = _parse_headers(header_blob.decode("utf-8", "ignore"))
        disposition = headers.get("content-disposition", "")
        if "form-data" not in disposition:
            continue

        params = {
            key.lower(): value
            for key, value in _DISPOSITION_PARAM_RE.findall(disposition)
        }
        name = params.get("name")
        if not name:
            raise MultipartError("Multipart part is missing a field name")

        parsed[name] = MultipartPart(
            name=name,
            filename=params.get("filename"),
            content_type=headers.get("content-type"),
            data=payload.rstrip(b"\r\n"),
        )

    if not parsed:
        raise MultipartError("No multipart fields were found")

    return parsed

def _parse_headers(header_blob: str) -> dict[str, str]:
    headers: dict[str, str] = {}
    for line in header_blob.splitlines():
        if ":" not in line:
            continue
        key, value = line.split(":", 1)
        headers[key.strip().lower()] = value.strip()
    return headers
```

▼ Show more

ReadFile

```
__init__.py
"""Resume parsing and scoring service."""

__all__ = ["__version__"]

__version__ = "0.1.0"
```

Thinking

Now let me read the extractors and remaining files:

ReadFile

```
__init__.py
from .base import ExtractionError, ExtractionResult, UnsupportedResumeError
from .sniff import extract_resume

__all__ = [
    "ExtractionError",
    "ExtractionResult",
    "UnsupportedResumeError",
    "extract_resume",
```

```
]

ReadFile

base.py
from __future__ import annotations

from dataclasses import dataclass, field

class ExtractionError(ValueError):
    """Raised when text extraction fails."""

class UnsupportedResumeError(ExtractionError):
    """Raised when a file type is unsupported."""

@dataclass(slots=True)
class ExtractionResult:
    file_type: str
    text: str
    lines: list[str]
    content_type: str | None = None
    detected_columns: int = 1
    metadata: dict[str, object] = field(default_factory=dict)

▼ Show more
ReadFile

pdf.py
from __future__ import annotations

import re
import zlib
from dataclasses import dataclass
from io import BytesIO

from .base import ExtractionResult

try:
    from pypdf import PdfReader
except Exception:  # pragma: no cover
    PdfReader = None

_OBJ_RE = re.compile(rb"(\d+)\s+(\d+)\s+obj\s*(.*?)\s*endobj", re.S)
_STREAM_RE = re.compile(rb"^(.*?)stream\r?\n(.*?)\r?\nendstream\s*$", re.S)
_BFCHAR_RE = re.compile(r"beginbfchar\s*(.*?)\s*endbfchar", re.S)
_BFRANGE_RE = re.compile(r"beginbfrange\s*(.*?)\s*endbfrange", re.S)
_TOKEN_RE = re.compile(r"\((?:\\.|[^\\)])*\)|<[0-9A-Fa-f]+>|/[A-Za-z0-9]+|-?\d*\.\d+|-?\d+|\[|\]|BT|
_STRING_TJ_RE = re.compile(r"\((?:\\.|[^\\)])*\)\s*Tj", re.S)
_STRING_TJ_ARRAY_RE = re.compile(r"\[(.*?)\]\s*TJ", re.S)
_PDF_STRING_RE = re.compile(r"\((?:\\.|[^\\)])*\)")

@dataclass(slots=True)
class _TextChunk:
    y: float
    x: float
    text: str

def extract_pdf(file_bytes: bytes) -> ExtractionResult:
    pypdf_result = _extract_with_pypdf(file_bytes)
    if pypdf_result is not None:
        layout_result = _extract_with_layout_parser(file_bytes)
        if layout_result is not None:
            pypdf_result.detected_columns = layout_result.detected_columns
            if not pypdf_result.text.strip() and layout_result.text.strip():
                return layout_result
        return pypdf_result

    layout_result = _extract_with_layout_parser(file_bytes)
    if layout_result is not None:
        return layout_result
    return ExtractionResult(
        file_type="pdf",
        text="",
        lines=[],
        content_type="application/pdf",
```

```
        detected_columns=1,
        metadata={"page_count": 0, "extractor_name": "custom-pdf"},
    )

def _extract_with_layout_parser(file_bytes: bytes) -> ExtractionResult | None:
    objects = _parse_objects(file_bytes)
    streams: dict[int, bytes] = {}
    font_maps = _build_font_maps(objects, streams)
    pages = _find_pages(objects)

    chunks: list[_TextChunk] = []
    x_positions: list[float] = []
    for page in pages:
        for content_id in page["content_ids"]:
            raw_stream = _get_stream(objects, streams, content_id)
            if not raw_stream:
                continue
            page_chunks = _extract_text_chunks(
                raw_stream.decode("latin1", "ignore"),
                page["fonts"],
                font_maps,
            )
            chunks.extend(page_chunks)
            x_positions.extend(chunk.x for chunk in page_chunks if chunk.text.strip())

    lines = _group_chunks(chunks)
    text = "\n".join(lines).strip()
    return ExtractionResult(
        file_type="pdf",
        text=text,
        lines=lines,
        content_type="application/pdf",
        detected_columns=_detect_columns(x_positions),
        metadata={"page_count": len(pages), "extractor_name": "custom-pdf"},
    )

def _extract_with_pypdf(file_bytes: bytes) -> ExtractionResult | None:
    if PdfReader is None:
        return None

    try:
        reader = PdfReader(BytesIO(file_bytes))
    except Exception:
        return None

    page_texts: list[str] = []
    for page in reader.pages:
        try:
            text = page.extract_text() or ""
        except Exception:
            text = ""
        cleaned = text.replace("\r", "\n").strip()
        if cleaned:
            page_texts.append(cleaned)

    if not page_texts:
        return None

    text = "\n\n".join(page_texts).strip()
    lines = [line.strip() for line in text.splitlines() if line.strip()]
    return ExtractionResult(
        file_type="pdf",
        text=text,
        lines=lines,
        content_type="application/pdf",
        detected_columns=1,
        metadata={"page_count": len(reader.pages), "extractor_name": "pypdf"},
    )

def _parse_objects(file_bytes: bytes) -> dict[int, bytes]:
    return {int(match.group(1)): match.group(3) for match in _OBJ_RE.finditer(file_bytes)}

def _get_stream(objects: dict[int, bytes], cache: dict[int, bytes], object_id: int) -> bytes | None:
    if object_id in cache:
        return cache[object_id]
```

```
    body = objects.get(object_id)
    if body is None:
        return None

    stream_match = _STREAM_RE.search(body)
    if not stream_match:
        return None

    raw = stream_match.group(2)
    try:
        data = zlib.decompress(raw)
    except zlib.error:
        data = raw

    cache[object_id] = data
    return data

def _build_font_maps(objects: dict[int, bytes], cache: dict[int, bytes]) -> dict[int, dict[int, str]]
    font_maps: dict[int, dict[int, str]] = {}
    for object_id, body in objects.items():
        decoded = body.decode("latin1", "ignore")
        to_unicode_match = re.search(r"/ToUnicode\s+(\d+)\s+0\s+R", decoded)
        if not to_unicode_match:
            continue
        cmap_stream = _get_stream(objects, cache, int(to_unicode_match.group(1)))
        if cmap_stream:
            font_maps[object_id] = _parse_cmap(cmap_stream.decode("latin1", "ignore"))
    return font_maps

def _parse_cmap(cmap_text: str) -> dict[int, str]:
    mapping: dict[int, str] = {}

    for block in _BFCHAR_RE.findall(cmap_text):
        for source, destination in re.findall(r"<([0-9A-Fa-f]+)>\s*<([0-9A-Fa-f]+)>", block):
            mapping[int(source, 16)] = bytes.fromhex(destination).decode("utf-16-be", "ignore")

    for block in _BFRANGE_RE.findall(cmap_text):
        for start_hex, end_hex, dst_hex in re.findall(
            r"<([0-9A-Fa-f]+)>\s*<([0-9A-Fa-f]+)>\s*<([0-9A-Fa-f]+)>",
            block,
        ):
            start_code = int(start_hex, 16)
            end_code = int(end_hex, 16)
            destination = int(dst_hex, 16)
            width = len(dst_hex)
            for offset, code in enumerate(range(start_code, end_code + 1)):
                mapping[code] = bytes.fromhex(f"{destination + offset:0{width}X}").decode("utf-16-be"

    return mapping

def _find_pages(objects: dict[int, bytes]) -> list[dict[str, object]]:
    pages: list[dict[str, object]] = []
    for body in objects.values():
        decoded = body.decode("latin1", "ignore")
        if "/Type /Page" not in decoded or "/Type /Pages" in decoded:
            continue
        content_ids = _extract_content_ids(decoded)
        if not content_ids:
            continue
        pages.append(
            {
                "content_ids": content_ids,
                "fonts": _extract_page_fonts(decoded, objects),
            }
        )
    return pages

def _extract_content_ids(page_body: str) -> list[int]:
    direct_match = re.findall(r"/Contents\s+(\d+)\s+0\s+R", page_body)
    if direct_match:
        return [int(match) for match in direct_match]
    array_match = re.search(r"/Contents\s*\[(.*?)\]", page_body, re.S)
    if not array_match:
        return []
    return [int(match) for match in re.findall(r"(\d+)\s+0\s+R", array_match.group(1))]
```

```
def _extract_page_fonts(page_body: str, objects: dict[int, bytes]) -> dict[str, int]:
    font_dict = _extract_font_dict_block(page_body)
    if font_dict:
        return {name: int(obj) for name, obj in re.findall(r"/([A-Za-z0-9]+)\s+(\d+)\s+0\s+R", font_

    resources_match = re.search(r"/Resources\s+(\d+)\s+0\s+R", page_body)
    if not resources_match:
        return {}

    resource_object = objects.get(int(resources_match.group(1)))
    if not resource_object:
        return {}

    resource_body = resource_object.decode("latin1", "ignore")
    font_dict = _extract_font_dict_block(resource_body)
    if not font_dict:
        return {}

    return {name: int(obj) for name, obj in re.findall(r"/([A-Za-z0-9]+)\s+(\d+)\s+0\s+R", font_dict

def _extract_font_dict_block(body: str) -> str | None:
    font_start = body.find("/Font")
    if font_start == -1:
        return None
    dict_start = body.find("<<", font_start)
    if dict_start == -1:
        return None

    depth = 0
    index = dict_start
    while index < len(body) - 1:
        pair = body[index : index + 2]
        if pair == "<<":
            depth += 1
            index += 2
            continue
        if pair == ">>":
            depth -= 1
            index += 2
            if depth == 0:
                return body[dict_start:index]
            continue
        index += 1
    return None

def _extract_text_chunks(
    content_stream: str,
    page_fonts: dict[str, int],
    font_maps: dict[int, dict[int, str]],
) -> list[_TextChunk]:
    tokens = _TOKEN_RE.findall(content_stream)

    chunks: list[_TextChunk] = []
    operands: list[object] = []
    in_array = False
    array_values: list[str] = []
    current_font_name: str | None = None
    x = 0.0
    y = 0.0

    for token in tokens:
        if token == "[":
            in_array = True
            array_values = []
            continue
        if token == "]":
            in_array = False
            operands.append(("array", array_values.copy()))
            continue
        if in_array:
            array_values.append(token)
            continue
        if re.fullmatch(r"-?\d*\.\d+|-?\d+", token):
            operands.append(float(token))
            continue
        if token.startswith("/F") or token.startswith("<") or token.startswith("("):
```

```
            operands.append(token)
            continue
        if token == "Tf":
            if len(operands) >= 2 and isinstance(operands[-2], str):
                current_font_name = operands[-2][1:]
            operands.clear()
            continue
        if token == "Tm":
            if len(operands) >= 6:
                x = float(operands[-2])
                y = float(operands[-1])
            operands.clear()
            continue
        if token in {"Td", "TD"}:
            if len(operands) >= 2:
                x += float(operands[-2])
                y += float(operands[-1])
            operands.clear()
            continue
        if token == "T*":
            y -= 12
            x = 0
            operands.clear()
            continue
        if token == "Tj":
            if operands and isinstance(operands[-1], str):
                text = _decode_operand(operands[-1], current_font_name, page_fonts, font_maps)
                if text.strip():
                    chunks.append(_TextChunk(y=y, x=x, text=text))
            operands.clear()
            continue
        if token == "TJ":
            if operands and isinstance(operands[-1], tuple):
                values = [
                    _decode_operand(item, current_font_name, page_fonts, font_maps)
                    for item in operands[-1][1]
                    if item.startswith("<") or item.startswith("(")
                ]
                joined = "".join(values).strip()
                if joined:
                    chunks.append(_TextChunk(y=y, x=x, text=joined))
            operands.clear()
            continue
        operands.clear()

    if not chunks:
        chunks.extend(_extract_literal_chunks(content_stream))
    return chunks

def _decode_operand(
    operand: str,
    current_font_name: str | None,
    fonts: dict[str, int],
    font_maps: dict[int, dict[int, str]],
) -> str:
    if operand.startswith("("):
        return _decode_literal_string(operand)
    if not operand.startswith("<"):
        return ""

    hex_text = operand[1:-1]
    if not hex_text:
        return ""

    font_object = fonts.get(current_font_name or "")
    cmap = font_maps.get(font_object, {}) if font_object else {}
    if not cmap:
        return _decode_hex_without_cmap(hex_text)
    step = 4 if len(hex_text) % 4 == 0 else 2
    decoded: list[str] = []
    for index in range(0, len(hex_text), step):
        code = int(hex_text[index : index + step], 16)
        decoded.append(cmap.get(code, _decode_hex_without_cmap(hex_text[index : index + step])))
    return "".join(decoded)

def _decode_hex_without_cmap(hex_text: str) -> str:
    try:
```

```
            if len(hex_text) % 4 == 0:
                decoded = bytes.fromhex(hex_text).decode("utf-16-be", "ignore")
                cleaned = "".join(ch for ch in decoded if ch.isprintable() or ch in "\n\t ")
                if cleaned.strip():
                    return cleaned
        except Exception:
            pass

        try:
            decoded = bytes.fromhex(hex_text).decode("latin1", "ignore")
            cleaned = "".join(ch for ch in decoded if ch.isprintable() or ch in "\n\t ")
            return cleaned
        except Exception:
            return ""

    def _decode_literal_string(literal: str) -> str:
        buffer: list[str] = []
        index = 1
        while index < len(literal) - 1:
            char = literal[index]
            if char != "\\":
                buffer.append(char)
                index += 1
                continue

            index += 1
            if index >= len(literal) - 1:
                break

            escaped = literal[index]
            escape_map = {
                "n": "\n",
                "r": "\r",
                "t": "\t",
                "b": "\b",
                "f": "\f",
                "\\": "\\",
                "(": "(",
                ")": ")",
            }
            if escaped in escape_map:
                buffer.append(escape_map[escaped])
                index += 1
                continue
            if escaped.isdigit():
                digits = escaped
                for _ in range(2):
                    if index + 1 < len(literal) - 1 and literal[index + 1].isdigit():
                        index += 1
                        digits += literal[index]
                buffer.append(chr(int(digits, 8)))
                index += 1
                continue
            buffer.append(escaped)
            index += 1
        return "".join(buffer)

    def _extract_literal_chunks(content_stream: str) -> list[_TextChunk]:
        chunks: list[_TextChunk] = []
        for index, match in enumerate(_STRING_TJ_RE.finditer(content_stream)):
            literal = match.group(0).rsplit("Tj", 1)[0].strip()
            text = _decode_literal_string(literal)
            if text.strip():
                chunks.append(_TextChunk(y=1000.0 - index, x=0.0, text=text))

        for index, match in enumerate(_STRING_TJ_ARRAY_RE.finditer(content_stream)):
            pieces = [_decode_literal_string(token) for token in _PDF_STRING_RE.findall(match.group(1))]
            text = "".join(pieces).strip()
            if text:
                chunks.append(_TextChunk(y=900.0 - index, x=0.0, text=text))
        return chunks

    def _group_chunks(chunks: list[_TextChunk]) -> list[str]:
        groups: list[dict[str, object]] = []
        for chunk in sorted(chunks, key=lambda item: (-item.y, item.x)):
            match_group = None
```

```
        for group in groups:
            if abs(group["y"] - chunk.y) <= 2.0:
                match_group = group
                break
        if match_group is None:
            match_group = {"y": chunk.y, "items": []}
            groups.append(match_group)
        match_group["items"].append((chunk.x, chunk.text))

    lines: list[str] = []
    for group in groups:
        ordered = ""
        last_x: float | None = None
        last_text = ""
        for x, text in sorted(group["items"], key=lambda item: item[0]):
            chunk = text.strip("\n")
            if not chunk:
                continue
            if ordered and last_x is not None:
                gap = x - last_x
                if gap > 14 and not ordered.endswith(" ") and not chunk.startswith(" "):
                    ordered += " "
            ordered += chunk
            last_x = x
            last_text = chunk
        ordered = re.sub(r"\s+", " ", ordered).strip()
        if ordered:
            lines.append(ordered)
    return lines

def _detect_columns(x_positions: list[float]) -> int:
    if not x_positions:
        return 1
    left = sum(1 for value in x_positions if value < 180)
    right = sum(1 for value in x_positions if value > 260)
    threshold = max(1, int(0.15 * len(x_positions)))
    return 2 if left >= threshold and right >= threshold else 1
▼ Show more
ReadFile

docx.py
from __future__ import annotations

import xml.etree.ElementTree as ET
from io import BytesIO
from zipfile import ZipFile

from .base import ExtractionError, ExtractionResult

_WORD_NS = {"w": "http://schemas.openxmlformats.org/wordprocessingml/2006/main"}

def extract_docx(file_bytes: bytes) -> ExtractionResult:
    try:
        archive = ZipFile(BytesIO(file_bytes))
    except Exception as exc:  # pragma: no cover
        raise ExtractionError("Failed to open DOCX archive") from exc

    document_parts = [
        name
        for name in archive.namelist()
        if name == "word/document.xml" or name.startswith("word/header") or name.startswith("word/fo
    ]
    if "word/document.xml" not in document_parts:
        raise ExtractionError("DOCX file is missing word/document.xml")

    ordered_parts = sorted(document_parts, key=_part_sort_key)
    lines: list[str] = []

    for part_name in ordered_parts:
        xml_bytes = archive.read(part_name)
        try:
            root = ET.fromstring(xml_bytes)
        except ET.ParseError as exc:
            raise ExtractionError(f"Failed to parse DOCX XML part: {part_name}") from exc

        lines.extend(_extract_part_lines(root))
```

```
    text = "\n".join(lines).strip()
    return ExtractionResult(
        file_type="docx",
        text=text,
        lines=lines,
        content_type="application/vnd.openxmlformats-officedocument.wordprocessingml.document",
        detected_columns=1,
        metadata={"parts": ordered_parts, "page_count": 1, "extractor_name": "docx-xml"},
    )

def _part_sort_key(part_name: str) -> tuple[int, str]:
    if part_name.startswith("word/header"):
        return (0, part_name)
    if part_name == "word/document.xml":
        return (1, part_name)
    return (2, part_name)

def _extract_part_lines(root: ET.Element) -> list[str]:
    body = root.find(".//w:body", _WORD_NS)
    if body is None:
        return []

    lines: list[str] = []
    for child in body:
        tag = _local_name(child.tag)
        if tag == "p":
            paragraph_text = _paragraph_text(child)
            if paragraph_text:
                lines.append(paragraph_text)
            continue
        if tag == "tbl":
            lines.extend(_table_lines(child))
    return lines

def _paragraph_text(paragraph: ET.Element) -> str:
    text = "".join(node.text or "" for node in paragraph.findall(".//w:t", _WORD_NS)).strip()
    return " ".join(text.split())

def _table_lines(table: ET.Element) -> list[str]:
    lines: list[str] = []
    for row in table.findall("./w:tr", _WORD_NS):
        row_cells: list[str] = []
        for cell in row.findall("./w:tc", _WORD_NS):
            cell_lines = [
                _paragraph_text(paragraph)
                for paragraph in cell.findall(".//w:p", _WORD_NS)
                if _paragraph_text(paragraph)
            ]
            if cell_lines:
                row_cells.append(" ".join(cell_lines))
        if row_cells:
            lines.append(" | ".join(row_cells))
    return lines

def _local_name(tag: str) -> str:
    return tag.rsplit("}", 1)[-1]
▼ Show more
ReadFile

sniff.py
from __future__ import annotations

from io import BytesIO
from pathlib import Path
from zipfile import BadZipFile, ZipFile

from .base import ExtractionError, ExtractionResult, UnsupportedResumeError
from .docx import extract_docx
from .pdf import extract_pdf

def extract_resume(file_bytes: bytes, filename: str, content_type: str | None = None) -> ExtractionR
    extension = Path(filename).suffix.lower()

    if file_bytes.startswith(b"%PDF"):
        result = extract_pdf(file_bytes)
```

```
    elif file_bytes.startswith(b"PK"):
        result = _extract_zip_container(file_bytes)
    elif file_bytes.startswith(bytes.fromhex("D0CF11E0")) or extension == ".doc":
        raise UnsupportedResumeError("Legacy .doc resumes are not supported in v1")
    else:
        raise UnsupportedResumeError("Unsupported file type. Only PDF and DOCX are supported.")

    if not result.text.strip():
        raise ExtractionError("The uploaded resume does not contain extractable text.")

    return ExtractionResult(
        file_type=result.file_type,
        text=result.text,
        lines=result.lines,
        content_type=content_type or result.content_type,
        detected_columns=result.detected_columns,
        metadata=result.metadata,
    )

def _extract_zip_container(file_bytes: bytes) -> ExtractionResult:
    try:
        archive = ZipFile(BytesIO(file_bytes))
    except BadZipFile as exc:
        raise ExtractionError("The uploaded archive is not a valid DOCX file.") from exc

    names = set(archive.namelist())
    if "[Content_Types].xml" in names and "word/document.xml" in names:
        return extract_docx(file_bytes)
    raise UnsupportedResumeError("ZIP containers must be valid DOCX files.")
▼ Show more
ReadFile

migrate_json_store.py
from __future__ import annotations

import argparse
import json

from .storage import init_storage, migrate_legacy_json_store

def main() -> None:
    parser = argparse.ArgumentParser(description="Migrate legacy JSON resume store into SQLite catalo
    parser.add_argument(
        "--retire-legacy-files",
        action="store_true",
        help="Move successfully migrated legacy JSON files to data/resumes_retired/<run_id>/.",
    )
    args = parser.parse_args()

    init_storage()
    result = migrate_legacy_json_store(retire_legacy_files=args.retire_legacy_files)
    print(json.dumps(result, indent=2))

if __name__ == "__main__":
    main()
▼ Show more
ReadFile

parsed_resumes.json
{
  "res_fresher_001": {
    "resume_id": "res_fresher_001",
    "candidate_type": "fresher",
    "normalized_resume": {
      "full_name": "Aarav Sharma",
      "emails": [
        "aarav.sharma@email.com"
      ],
      "phones": [
        "+919876543210"
      ],
      "current_location": "Hyderabad, Telangana",
      "links": [
        "https://www.linkedin.com/in/aaravsharma"
      ],
      "skills": [
```

```
        "Data Structures",
        "Docker",
        "FastAPI",
        "Git",
        "Python",
        "REST APIs",
        "SQL"
      ],
      "experience_summary": "Entry-level Python developer with strong foundations in backend APIs and
      "employment_history": [
        {
          "company": "TechNova Labs",
          "title": "Backend Intern",
          "start_date": "Jan 2025",
          "end_date": "Mar 2025",
          "employment_type": "internship",
          "location": null,
          "description": "Built REST APIs in FastAPI and wrote SQL queries for recruiter workflows.",
          "is_current": false
        }
      ],
      "education": [
        {
          "degree": "B.Tech",
          "specialization": null,
          "institution": null,
          "completion_year": 2025,
          "score": "8.4"
        }
      ],
      "current_role": "Backend Intern",
      "notice_period": "Immediate",
      "current_ctc": null,
      "expected_ctc": null,
      "preferred_location": "Hyderabad",
      "work_authorization": null
    },
    "parser_metadata": {
      "filename": "resume.pdf",
      "file_type": "pdf",
      "content_type": "application/pdf",
      "file_size_bytes": 1498,
      "line_count": 18,
      "section_order": [
        "header",
        "summary",
        "skills",
        "education",
        "internships",
        "projects",
        "achievements"
      ],
      "detected_columns": 1,
      "is_scanned": false,
      "sensitive_findings": [],
      "extracted_text": "Aarav Sharma\nHyderabad, Telangana\n+91 98765 43210 | aarav.sharma@email.com
    },
    "resume_quality": {
      "score": 100,
      "breakdown": {
        "completeness": 40.0,
        "structure": 25.0,
        "contactability": 20.0,
        "signal_quality": 15.0
      },
      "recommendation": "STRONG",
      "summary": "Resume quality 100/100. Strengths: 7 skills extracted, education parsed, contact de
    },
    "resume_text": "Aarav Sharma\nHyderabad, Telangana\n+91 98765 43210 | aarav.sharma@email.com | ww
  },
  "res_lateral_001": {
    "resume_id": "res_lateral_001",
    "candidate_type": "lateral",
    "normalized_resume": {
      "full_name": "Priya Nair",
      "emails": [
        "priya.nair@sample.com"
```

```
  ],
  "phones": [
    "+919988766554"
  ],
  "current_location": "Bengaluru, Karnataka",
  "links": [
    "https://github.com/priyanair",
    "https://www.linkedin.com/in/priyanair"
  ],
  "skills": [
    "AWS",
    "Docker",
    "Git",
    "Java",
    "Kubernetes",
    "PostgreSQL",
    "REST APIs",
    "SQL",
    "Spring",
    "Spring Boot"
  ],
  "experience_summary": "Backend engineer with 4.2 years of experience building hiring platforms
  "employment_history": [
    {
      "company": "Horizon Stack",
      "title": "Senior Software Engineer",
      "start_date": "Jun 2023",
      "end_date": "Present",
      "employment_type": "experience",
      "location": "Bengaluru",
      "description": "Built recruiter APIs, Spring Boot services, and PostgreSQL-backed workflows
      "is_current": true
    },
    {
      "company": "BlueOrbit Systems",
      "title": "Software Engineer",
      "start_date": "Jul 2021",
      "end_date": "May 2023",
      "employment_type": "experience",
      "location": "Pune",
      "description": "Delivered Java microservices and AWS integrations.",
      "is_current": false
    }
  ],
  "education": [
    {
      "degree": "B.E",
      "specialization": null,
      "institution": "PES University",
      "completion_year": 2020,
      "score": "8.1"
    }
  ],
  "current_role": "Senior Software Engineer",
  "notice_period": "30 Days",
  "current_ctc": "16 LPA",
  "expected_ctc": "22 LPA",
  "preferred_location": "Bengaluru",
  "work_authorization": "India"
},
"parser_metadata": {
  "filename": "resume.docx",
  "file_type": "docx",
  "content_type": "application/vnd.openxmlformats-officedocument.wordprocessingml.document",
  "file_size_bytes": 1478,
  "line_count": 21,
  "section_order": [
    "header",
    "summary",
    "skills",
    "experience",
    "education"
  ],
  "detected_columns": 1,
  "is_scanned": false,
  "sensitive_findings": [
    "aadhaar",
```

```
      "pan"
    ],
    "extracted_text": "Priya Nair\nBengaluru, Karnataka\n+91 99887 66554 | priya.nair@sample.com |
  },
  "resume_quality": {
    "score": 100,
    "breakdown": {
      "completeness": 40.0,
      "structure": 25.0,
      "contactability": 20.0,
      "signal_quality": 15.0
    },
    "recommendation": "STRONG",
    "summary": "Resume quality 100/100. Strengths: 10 skills extracted, education parsed, contact d
  },
  "resume_text": "Priya Nair\nBengaluru, Karnataka\n+91 99887 66554 | priya.nair@sample.com | https
},
"res_123": {
  "resume_id": "res_123",
  "candidate_type": "lateral",
  "normalized_resume": {
    "full_name": "Bankuru Ganesh",
    "emails": [
      "bankuruganesh143@gmail.com"
    ],
    "phones": [
      "+917670871356"
    ],
    "current_location": "pedabondapalli, parvathipuram manyam, Andhra Pradesh, 535501",
    "links": [
      "https://github.com/ganeshbankuru",
      "https://www.linkedin.com/in/ganesh-bankuru-0386b1253"
    ],
    "skills": [
      "AWS",
      "Azure",
      "CSS",
      "Docker",
      "Flask",
      "Git",
      "HTML",
      "Java",
      "JavaScript",
      "PostgreSQL",
      "Python",
      "SQL"
    ],
    "experience_summary": "Motivated Information Technology undergraduate with strong hands-on expe
    "employment_history": [],
    "education": [
      {
        "degree": "B.Tech",
        "specialization": null,
        "institution": "Anil Neerukonda Institute of Technology",
        "completion_year": 2026,
        "score": null
      },
      {
        "degree": "Intermediate",
        "specialization": null,
        "institution": "Sri Viswa Junior College",
        "completion_year": 2022,
        "score": null
      },
      {
        "degree": "SSC",
        "specialization": null,
        "institution": "Suresh High School (2020)",
        "completion_year": 2020,
        "score": null
      }
    ],
    "current_role": "Motivated Information Technology Undergraduate",
    "notice_period": null,
    "current_ctc": null,
    "expected_ctc": null,
    "preferred_location": null,
```

```
      "work_authorization": null
    },
    "parser_metadata": {
      "filename": "A22126511005_BankuruGanesh.pdf",
      "file_type": "pdf",
      "content_type": "application/pdf",
      "file_size_bytes": 658657,
      "line_count": 45,
      "section_order": [
        "header",
        "summary",
        "education",
        "skills",
        "projects",
        "certifications"
      ],
      "detected_columns": 1,
      "is_scanned": false,
      "sensitive_findings": [],
      "extracted_text": "Bankuru Ganesh \npedabondapalli, parvathipuram manyam, Andhra Pradesh, 5355
    },
    "resume_quality": {
      "score": 98,
      "breakdown": {
        "completeness": 40.0,
        "structure": 25.0,
        "contactability": 20.0,
        "signal_quality": 13.0
      },
      "recommendation": "STRONG",
      "summary": "Resume quality 98/100. Strengths: 12 skills extracted, education parsed, contact d
    },
    "resume_text": "Bankuru Ganesh \npedabondapalli, parvathipuram manyam, Andhra Pradesh, 535501 | +
  }
}
```

▼ Show more

Thinking
Now let me check the project-level configuration and test files:

ReadFile

pyproject.toml
```
[build-system]
requires = ["setuptools>=68", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "resume-service"
version = "0.1.0"
description = "Internal Python microservice for Indian resume parsing and scoring."
requires-python = ">=3.11"
dependencies = [
  "fastapi>=0.121.0,<1.0.0",
  "httpx>=0.28.0,<1.0.0",
  "pydantic>=2.11.0,<3.0.0",
  "pypdf>=5.0.0,<6.0.0",
  "uvicorn>=0.38.0,<1.0.0",
]

[tool.setuptools.packages.find]
include = ["resume_service*"]
```
▼ Show more
ReadFile

test_parser.py
```
from __future__ import annotations

import unittest
from pathlib import Path

from resume_service.extractors import UnsupportedResumeError
from resume_service.parser import parse_resume_document
from resume_service.schemas import CandidateType

from resume_service.tests.fixture_builders import build_docx, build_legacy_doc_stub, build_pdf, load_
```

```
class ParserTests(unittest.TestCase):
    def test_parse_pdf_fresher_resume(self) -> None:
        case = load_case("fresher_resume")
        pdf_bytes = build_pdf(case["lines"])

        response = parse_resume_document(
            file_bytes=pdf_bytes,
            filename="fresher.pdf",
            resume_id=case["resume_id"],
            candidate_type=CandidateType(case["candidate_type"]),
            content_type="application/pdf",
        )

        self.assertEqual(response.normalized_resume.full_name, "Aarav Sharma")
        self.assertIn("aarav.sharma@email.com", response.normalized_resume.emails)
        self.assertIn("+919876543210", response.normalized_resume.phones)
        self.assertIn("Python", response.normalized_resume.skills)
        self.assertIn("FastAPI", response.normalized_resume.skills)
        self.assertEqual(response.normalized_resume.notice_period, "Immediate")
        self.assertEqual(response.normalized_resume.preferred_location, "Hyderabad")
        self.assertEqual(response.parser_metadata.file_type, "pdf")
        self.assertEqual(response.parser_metadata.extractor_name, "pypdf")
        self.assertIsNotNone(response.parser_metadata.content_fingerprint)
        self.assertTrue(response.normalized_resume.projects)
        self.assertTrue(response.normalized_resume.skill_evidence)
        self.assertGreater(response.resume_quality.score, 0)
        self.assertIn(response.resume_quality.recommendation, {"STRONG", "REVIEW", "IMPROVE"})

    def test_parse_docx_lateral_resume_and_redact_sensitive(self) -> None:
        case = load_case("lateral_resume")
        docx_bytes = build_docx(case["lines"])

        response = parse_resume_document(
            file_bytes=docx_bytes,
            filename="lateral.docx",
            resume_id=case["resume_id"],
            candidate_type=CandidateType(case["candidate_type"]),
            content_type="application/vnd.openxmlformats-officedocument.wordprocessingml.document",
        )

        self.assertEqual(response.normalized_resume.full_name, "Priya Nair")
        self.assertEqual(response.normalized_resume.current_role, "Senior Software Engineer")
        self.assertEqual(response.normalized_resume.current_ctc, "16 LPA")
        self.assertEqual(response.normalized_resume.expected_ctc, "22 LPA")
        self.assertEqual(response.normalized_resume.work_authorization, "India")
        self.assertIn("pan", response.parser_metadata.sensitive_findings)
        self.assertIn("aadhaar", response.parser_metadata.sensitive_findings)
        self.assertTrue(response.warnings)
        self.assertGreater(response.resume_quality.score, 0)

    def test_legacy_doc_is_rejected(self) -> None:
        with self.assertRaises(UnsupportedResumeError):
            parse_resume_document(
                file_bytes=build_legacy_doc_stub(),
                filename="legacy.doc",
                resume_id="res_legacy",
                candidate_type=CandidateType.lateral,
                content_type="application/msword",
            )

    def test_scanned_pdf_is_rejected(self) -> None:
        with self.assertRaises(UnsupportedResumeError):
            parse_resume_document(
                file_bytes=build_pdf([], image_only=True),
                filename="scan.pdf",
                resume_id="res_scan",
                candidate_type=CandidateType.lateral,
                content_type="application/pdf",
            )

    def test_spring_boot_does_not_also_extract_plain_spring(self) -> None:
        response = parse_resume_document(
            file_bytes=build_docx(
                [
                    "Riya Verma",
```

```
                "riya@example.com | +91 9876543210 | Hyderabad",
                "Skills",
                "Spring Boot, Java, PostgreSQL",
                "Education",
                "B.Tech - Example Institute (2024)",
            ]
        ),
        filename="spring-boot.docx",
        resume_id="res_spring_boot",
        candidate_type=CandidateType.lateral,
        content_type="application/vnd.openxmlformats-officedocument.wordprocessingml.document",
    )

    self.assertIn("Spring Boot", response.normalized_resume.skills)
    self.assertNotIn("Spring", response.normalized_resume.skills)

def test_docx_table_content_is_extracted(self) -> None:
    response = parse_resume_document(
        file_bytes=build_docx(
            [
                "Meera Iyer",
                "meera@example.com | +91 9876543210 | Chennai, Tamil Nadu",
                "Education",
                "B.Tech - Example Institute (2024)",
                "Certifications",
            ],
            table_rows=[["Skills", "Terraform, AWS, Docker"], ["Certifications", "AWS Certified D
        ),
        filename="table.docx",
        resume_id="res_table_docx",
        candidate_type=CandidateType.lateral,
        content_type="application/vnd.openxmlformats-officedocument.wordprocessingml.document",
    )

    self.assertIn("Terraform", response.normalized_resume.skills)
    self.assertIn("AWS", response.normalized_resume.skills)
    self.assertTrue(response.normalized_resume.certifications)

def test_real_repo_sample_pdf_parses(self) -> None:
    sample_path = Path(__file__).resolve().parents[2] / "A22126511005_BankuruGanesh.pdf"
    response = parse_resume_document(
        file_bytes=sample_path.read_bytes(),
        filename=sample_path.name,
        resume_id="res_real_pdf",
        candidate_type=CandidateType.lateral,
        content_type="application/pdf",
    )

    self.assertEqual(response.normalized_resume.full_name, "Bankuru Ganesh")
    self.assertGreaterEqual(response.parser_metadata.page_count, 1)
    self.assertTrue(response.normalized_resume.projects)

def test_two_column_pdf_sets_detected_columns(self) -> None:
    response = parse_resume_document(
        file_bytes=build_pdf(
            ["Rohit Das", "Hyderabad, Telangana", "Skills", "Python, FastAPI"],
            right_column_lines=["Projects", "Talent Portal | Python, SQL"],
        ),
        filename="two-column.pdf",
        resume_id="res_two_column",
        candidate_type=CandidateType.fresher,
        content_type="application/pdf",
    )

    self.assertGreaterEqual(response.parser_metadata.detected_columns, 2)

def test_multi_page_pdf_includes_all_pages(self) -> None:
    response = parse_resume_document(
        file_bytes=build_pdf(
            [],
            pages=[
                ["Ananya Rao", "Bengaluru, Karnataka", "Projects", "Parser App | Python, FastAPI"
                ["Certifications", "AWS Certified Developer - 2024", "Education", "B.Tech - Examp
            ],
        ),
        filename="multi-page.pdf",
```

```
            resume_id="res_multi_page",
            candidate_type=CandidateType.fresher,
            content_type="application/pdf",
        )

        self.assertGreaterEqual(response.parser_metadata.page_count, 2)
        self.assertTrue(response.normalized_resume.projects)
        self.assertTrue(response.normalized_resume.certifications)

    def test_pan_false_positive_reduced(self) -> None:
        response = parse_resume_document(
            file_bytes=build_docx(
                [
                    "Vikram Singh",
                    "vikram@example.com | +91 9876543210 | Pune, Maharashtra",
                    "Summary",
                    "Worked with internal code ABCDE1234F in a billing platform.",
                    "Education",
                    "B.Tech - Example Institute (2023)",
                ]
            ),
            filename="pan-false-positive.docx",
            resume_id="res_pan_false_positive",
            candidate_type=CandidateType.lateral,
            content_type="application/vnd.openxmlformats-officedocument.wordprocessingml.document",
        )

        self.assertNotIn("pan", response.parser_metadata.sensitive_findings)
▼ Show more
ReadFile

test_scoring.py
from __future__ import annotations

import unittest

from resume_service.parser import parse_resume_document
from resume_service.schemas import CandidateType, JobContext, ScoreResumeRequest
from resume_service.scoring import score_resume

from resume_service.tests.fixture_builders import build_docx, build_pdf, load_case

class ScoringTests(unittest.TestCase):
    def test_exactish_match_scores_high(self) -> None:
        case = load_case("lateral_resume")
        parsed = parse_resume_document(
            file_bytes=build_docx(case["lines"]),
            filename="lateral.docx",
            resume_id=case["resume_id"],
            candidate_type=CandidateType(case["candidate_type"]),
        )

        response = score_resume(
            ScoreResumeRequest(
                resume_id=case["resume_id"],
                normalized_resume=parsed.normalized_resume,
                resume_text=parsed.parser_metadata.extracted_text,
                job_context=JobContext(
                    job_id="job_backend_001",
                    title="Senior Backend Developer",
                    description="Looking for Java, Spring Boot, REST APIs, PostgreSQL, AWS and Docke
                    required_skills=["Java", "Spring Boot", "REST APIs", "PostgreSQL", "AWS"],
                    preferred_skills=["Docker", "Kubernetes"],
                    experience_min_years=3,
                    experience_max_years=6,
                    location="Bengaluru",
                ),
            )
        )

        self.assertGreaterEqual(response.jd_match_score, 80)
        self.assertGreaterEqual(response.resume_quality_score, 60)
        self.assertEqual(response.recommendation, "SHORTLIST")
        self.assertIn("AWS", response.matched_skills)

    def test_missing_skills_are_reported(self) -> None:
```

```
        case = load_case("fresher_resume")
        parsed = parse_resume_document(
            file_bytes=build_pdf(case["lines"]),
            filename="fresher.pdf",
            resume_id=case["resume_id"],
            candidate_type=CandidateType(case["candidate_type"]),
        )

        response = score_resume(
            ScoreResumeRequest(
                resume_id=case["resume_id"],
                normalized_resume=parsed.normalized_resume,
                resume_text=parsed.parser_metadata.extracted_text,
                job_context=JobContext(
                    job_id="job_data_001",
                    title="Cloud Data Engineer",
                    description="Need Python, AWS, Kubernetes and PostgreSQL for a cloud-heavy data
                    required_skills=["Python", "AWS", "Kubernetes", "PostgreSQL"],
                    preferred_skills=["Docker"],
                    experience_min_years=2,
                ),
            )
        )

        self.assertIn("AWS", response.missing_skills)
        self.assertIn("Kubernetes", response.missing_skills)
        self.assertLess(response.jd_match_score, 80)
        self.assertIn(response.recommendation, {"REVIEW", "REJECT"})
▼ Show more
ReadFile

test_contracts.py
from __future__ import annotations

import unittest

from fastapi.testclient import TestClient

from resume_service.main import app
from resume_service.storage import DB_PATH, load_parsed_resume

from resume_service.tests.fixture_builders import build_docx, build_legacy_doc_stub, build_pdf, load_

class ContractTests(unittest.TestCase):
    @classmethod
    def setUpClass(cls) -> None:
        cls.client = TestClient(app)

    def test_parse_endpoint_accepts_multipart_pdf(self) -> None:
        case = load_case("fresher_resume")
        resume_id = "contract_parse_pdf_001"
        response = self.client.post(
            "/v1/parse",
            files={"file": ("resume.pdf", build_pdf(case["lines"]), "application/pdf")},
            data={"resume_id": resume_id, "candidate_type": case["candidate_type"]},
        )

        self.assertEqual(response.status_code, 200)
        body = response.json()
        self.assertEqual(body["resume_id"], resume_id)
        self.assertEqual(body["candidate_type"], case["candidate_type"])
        self.assertEqual(body["normalized_resume"]["full_name"], "Aarav Sharma")
        self.assertIn("resume_quality", body)
        self.assertIn("score", body["resume_quality"])
        self.assertTrue(DB_PATH.exists())
        self.assertIsNotNone(load_parsed_resume(resume_id))

    def test_score_endpoint_returns_breakdown(self) -> None:
        case = load_case("lateral_resume")
        resume_id = "contract_score_breakdown_001"
        parse_response = self.client.post(
            "/v1/parse",
            files={
                "file": (
                    "resume.docx",
                    build_docx(case["lines"]),
```

```
                    "application/vnd.openxmlformats-officedocument.wordprocessingml.document",
                )
            },
            data={"resume_id": resume_id, "candidate_type": case["candidate_type"]},
        )
        self.assertEqual(parse_response.status_code, 200)

        score_response = self.client.post(
            "/v1/score",
            json={
                "resume_id": resume_id,
                "job_context": {
                    "job_id": "job_backend_001",
                    "title": "Senior Backend Developer",
                    "description": "Java Spring Boot AWS PostgreSQL REST APIs",
                    "required_skills": ["Java", "Spring Boot", "REST APIs", "PostgreSQL"],
                    "preferred_skills": ["AWS", "Docker"],
                    "experience_min_years": 3
                }
            },
        )

        self.assertEqual(score_response.status_code, 200)
        body = score_response.json()
        self.assertEqual(
            set(body.keys()),
            {
                "resume_id",
                "jd_match_score",
                "resume_quality_score",
                "breakdown",
                "matched_skills",
                "missing_skills",
                "recommendation",
                "warnings",
                "summary",
            },
        )
        self.assertIn("jd_match", body["breakdown"])
        self.assertIn("resume_quality", body["breakdown"])
        self.assertIn("recommendation", body)

    def test_score_endpoint_uses_stored_resume_data_with_minimal_payload(self) -> None:
        case = load_case("fresher_resume")
        resume_id = "contract_storage_score_001"
        parse_response = self.client.post(
            "/v1/parse",
            files={"file": ("resume.pdf", build_pdf(case["lines"]), "application/pdf")},
            data={"resume_id": resume_id, "candidate_type": case["candidate_type"]},
        )
        self.assertEqual(parse_response.status_code, 200)

        score_response = self.client.post(
            "/v1/score",
            json={
                "resume_id": resume_id,
                "job_context": {
                    "job_id": "job_python_001",
                    "title": "Python Intern",
                    "description": "Need Python, FastAPI, SQL and Docker.",
                    "required_skills": ["Python", "SQL"],
                    "preferred_skills": ["FastAPI", "Docker"],
                },
            },
        )

        self.assertEqual(score_response.status_code, 200)
        body = score_response.json()
        self.assertEqual(body["resume_id"], resume_id)
        self.assertIn("Python", body["matched_skills"])

    def test_score_endpoint_returns_404_when_resume_not_stored(self) -> None:
        response = self.client.post(
            "/v1/score",
            json={
                "resume_id": "missing_resume_contract_404",
                "job_context": {
```

```
                "job_id": "job_missing_001",
                "title": "Backend Intern",
                "required_skills": ["Python"],
            },
        },
    )

    self.assertEqual(response.status_code, 404)

def test_parse_endpoint_rejects_legacy_doc(self) -> None:
    response = self.client.post(
        "/v1/parse",
        files={"file": ("legacy.doc", build_legacy_doc_stub(), "application/msword")},
        data={"resume_id": "contract_legacy_doc_001", "candidate_type": "lateral"},
    )

    self.assertEqual(response.status_code, 422)
    self.assertIn("not supported", response.json()["message"].lower())

def test_parse_endpoint_rejects_corrupted_payload(self) -> None:
    response = self.client.post(
        "/v1/parse",
        files={"file": ("broken.pdf", b"not-a-real-pdf", "application/pdf")},
        data={"resume_id": "res_broken", "candidate_type": "lateral"},
    )

    self.assertEqual(response.status_code, 422)

def test_parse_endpoint_requires_file_field(self) -> None:
    response = self.client.post(
        "/v1/parse",
        files={"placeholder": ("note.txt", b"placeholder", "text/plain")},
        data={"resume_id": "contract_missing_file_001", "candidate_type": "lateral"},
    )

    self.assertEqual(response.status_code, 422)
    self.assertEqual(response.json()["detail"], "Multipart field 'file' is required.")

def test_parse_endpoint_requires_resume_id_field(self) -> None:
    response = self.client.post(
        "/v1/parse",
        files={"file": ("resume.pdf", build_pdf(["Example Resume"]), "application/pdf")},
        data={"candidate_type": "lateral"},
    )

    self.assertEqual(response.status_code, 422)
    self.assertEqual(response.json()["detail"], "Multipart field 'resume_id' is required.")

def test_parse_endpoint_returns_expected_response_shape(self) -> None:
    case = load_case("fresher_resume")
    resume_id = "contract_parse_shape_001"
    response = self.client.post(
        "/v1/parse",
        files={"file": ("resume.pdf", build_pdf(case["lines"]), "application/pdf")},
        data={"resume_id": resume_id, "candidate_type": case["candidate_type"]},
    )

    self.assertEqual(response.status_code, 200)
    body = response.json()
    self.assertEqual(
        set(body.keys()),
        {
            "resume_id",
            "candidate_type",
            "normalized_resume",
            "warnings",
            "missing_fields",
            "resume_quality",
            "parser_metadata",
        },
    )
    self.assertEqual(
        set(body["resume_quality"].keys()),
        {"score", "breakdown", "recommendation", "summary"},
    )
    self.assertIn("projects", body["normalized_resume"])
```

```
        self.assertIn("certifications", body["normalized_resume"])
        self.assertIn("skill_evidence", body["normalized_resume"])
        self.assertIn("extracted_text", body["parser_metadata"])
        self.assertIn("extractor_name", body["parser_metadata"])
        self.assertIn("page_count", body["parser_metadata"])
        self.assertIn("content_fingerprint", body["parser_metadata"])
```

▼ Show more

ReadFile

```
fixture_builders.py
from __future__ import annotations

import json
from io import BytesIO
from pathlib import Path
from zipfile import ZIP_DEFLATED, ZipFile

FIXTURE_DIR = Path(__file__).parent / "fixtures"

def load_case(name: str) -> dict:
    return json.loads((FIXTURE_DIR / f"{name}.json").read_text(encoding="utf-8"))

def build_docx(lines: list[str], *, table_rows: list[list[str]] | None = None) -> bytes:
    content_types = """<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<Types xmlns="http://schemas.openxmlformats.org/package/2006/content-types">
  <Default Extension="rels" ContentType="application/vnd.openxmlformats-package.relationships+xml"/>
  <Default Extension="xml" ContentType="application/xml"/>
  <Override PartName="/word/document.xml" ContentType="application/vnd.openxmlformats-officedocument
</Types>
"""
    rels = """<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<Relationships xmlns="http://schemas.openxmlformats.org/package/2006/relationships">
  <Relationship Id="rId1" Type="http://schemas.openxmlformats.org/officeDocument/2006/relationships/
</Relationships>
"""
    paragraphs = "".join(
        f"<w:p><w:r><w:t>{_xml_escape(line)}</w:t></w:r></w:p>"
        for line in lines
    )
    table_xml = ""
    if table_rows:
        table_xml = "<w:tbl>" + "".join(
            "<w:tr>"
            + "".join(f"<w:tc><w:p><w:r><w:t>{_xml_escape(cell)}</w:t></w:r></w:p></w:tc>" for cell i
            + "</w:tr>"
            for row in table_rows
        ) + "</w:tbl>"
    document = f"""<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<w:document xmlns:w="http://schemas.openxmlformats.org/wordprocessingml/2006/main">
  <w:body>
    {paragraphs}
    {table_xml}
  </w:body>
</w:document>
"""
    buffer = BytesIO()
    with ZipFile(buffer, "w", compression=ZIP_DEFLATED) as archive:
        archive.writestr("[Content_Types].xml", content_types)
        archive.writestr("_rels/.rels", rels)
        archive.writestr("word/document.xml", document)
    return buffer.getvalue()

def build_pdf(lines: list[str], *, image_only: bool = False, pages: list[list[str]] | None = None, ri
    if image_only:
        page_streams = ["q 1 0 0 1 0 0 cm 0 0 100 100 re S Q"]
    else:
        page_sets = pages or [lines]
        page_streams = [_build_pdf_page(page_lines, right_column_lines if index == 0 else None) for i

    objects: list[bytes] = [b"<< /Type /Catalog /Pages 2 0 R >>"]
    page_object_ids: list[int] = []
    stream_object_ids: list[int] = []
    font_object_id = 3 + (len(page_streams) * 2)
    kids: list[str] = []

    current_id = 3
```

```
    for stream_text in page_streams:
        page_object_id = current_id
        stream_object_id = current_id + 1
        page_object_ids.append(page_object_id)
        stream_object_ids.append(stream_object_id)
        kids.append(f"{page_object_id} 0 R")
        current_id += 2

    objects.append(f"<< /Type /Pages /Kids [{' '.join(kids)}] /Count {len(page_streams)} >>".encode('
    for page_index, (page_object_id, stream_object_id) in enumerate(zip(page_object_ids, stream_objec
        objects.append(
            f"<< /Type /Page /Parent 2 0 R /MediaBox [0 0 612 792] /Contents {stream_object_id} 0 R /
        )
        stream = page_streams[page_index].encode("latin1", "ignore")
        objects.append(f"<< /Length {len(stream)} >>\nstream\n".encode("latin1") + stream + b"\nendst
    objects.append(b"<< /Type /Font /Subtype /Type1 /BaseFont /Helvetica >>")

    pdf = bytearray(b"%PDF-1.4\n")
    offsets = [0]
    for index, obj in enumerate(objects, start=1):
        offsets.append(len(pdf))
        pdf.extend(f"{index} 0 obj\n".encode("latin1"))
        pdf.extend(obj)
        pdf.extend(b"\nendobj\n")

    xref_start = len(pdf)
    pdf.extend(f"xref\n0 {len(objects) + 1}\n".encode("latin1"))
    pdf.extend(b"0000000000 65535 f \n")
    for offset in offsets[1:]:
        pdf.extend(f"{offset:010} 00000 n \n".encode("latin1"))
    pdf.extend(
        f"trailer\n<< /Size {len(objects) + 1} /Root 1 0 R >>\nstartxref\n{xref_start}\n%%EOF".encode
    )
    return bytes(pdf)

def build_legacy_doc_stub() -> bytes:
    return bytes.fromhex("D0CF11E0A1B11AE1") + b"legacy-doc"

def _build_pdf_page(lines: list[str], right_column_lines: list[str] | None = None) -> str:
    commands = ["BT", "/F1 11 Tf", "40 780 Td"]
    first = True
    for line in lines:
        if not first:
            commands.append("0 -15 Td")
        commands.append(f"({_pdf_escape(line)}) Tj")
        first = False
    commands.append("ET")
    if right_column_lines:
        commands.extend(["BT", "/F1 11 Tf", "320 780 Td"])
        first = True
        for line in right_column_lines:
            if not first:
                commands.append("0 -15 Td")
            commands.append(f"({_pdf_escape(line)}) Tj")
            first = False
        commands.append("ET")
    return "\n".join(commands)

def _xml_escape(value: str) -> str:
    return (
        value.replace("&", "&amp;")
        .replace("<", "&lt;")
        .replace(">", "&gt;")
    )

def _pdf_escape(value: str) -> str:
    return value.replace("\\", "\\\\").replace("(", "\\(").replace(")", "\\)")
```
▼ Show more

Thinking
Now let me check the test fixtures and the critical flows test:

ReadFile

fresher_resume.json
```
{
```

```
  "candidate_type": "fresher",
  "resume_id": "res_fresher_001",
  "lines": [
    "Aarav Sharma",
    "Hyderabad, Telangana",
    "+91 98765 43210 | aarav.sharma@email.com | www.linkedin.com/in/aaravsharma | github.com/aarav",
    "Career Objective",
    "Entry-level Python developer with strong foundations in backend APIs and data structures.",
    "Technical Skills",
    "Python, FastAPI, SQL, Git, REST APIs, Data Structures, Docker",
    "Education",
    "B.Tech in Computer Science | JNTU Hyderabad | 2025 | CGPA 8.4",
    "Internships",
    "Backend Intern | TechNova Labs | Jan 2025 - Mar 2025",
    "Built REST APIs in FastAPI and wrote SQL queries for recruiter workflows.",
    "Projects",
    "Resume Analyzer | Python, FastAPI, SQL",
    "Achievements",
    "Finalist - Smart India Hackathon",
    "Notice Period: Immediate",
    "Preferred Location: Hyderabad, Bengaluru"
  ]
}
```

▼ Show more
ReadFile

lateral_resume.json

```
{
  "candidate_type": "lateral",
  "resume_id": "res_lateral_001",
  "lines": [
    "Priya Nair",
    "Bengaluru, Karnataka",
    "+91 99887 66554 | priya.nair@sample.com | https://www.linkedin.com/in/priyanair | https://githul
    "Professional Summary",
    "Backend engineer with 4.2 years of experience building hiring platforms and recruiter workflows
    "Technical Skills",
    "Java, Spring Boot, REST APIs, SQL, PostgreSQL, AWS, Docker, Kubernetes, Git",
    "Professional Experience",
    "Senior Software Engineer | Horizon Stack | Jun 2023 - Present | Bengaluru",
    "Built recruiter APIs, Spring Boot services, and PostgreSQL-backed workflows.",
    "Software Engineer | BlueOrbit Systems | Jul 2021 - May 2023 | Pune",
    "Delivered Java microservices and AWS integrations.",
    "Education",
    "B.E in Information Science | PES University | 2020 | CGPA 8.1",
    "Current CTC: 16 LPA",
    "Expected CTC: 22 LPA",
    "Notice Period: 30 Days",
    "Work Authorization: India",
    "Preferred Location: Bengaluru, Hyderabad",
    "PAN: ABCDE1234F",
    "Aadhaar: 1234 5678 9012"
  ]
}
```

▼ Show more
ReadFile

test_critical_flows.py
Showing lines 1-624 of 703 total lines.

```
from __future__ import annotations

import json
import tempfile
import unittest
from pathlib import Path
from unittest import mock
from uuid import uuid4

from fastapi.testclient import TestClient

from resume_service.main import app
from resume_service.normalization import (
```

```
    _is_missing,
    _normalize_phone,
    _split_sections,
)
from resume_service.parser import parse_resume_document
from resume_service.schemas import CandidateType, JobContext, ScoreResumeRequest
from resume_service.scoring import (
    _canonicalize_skills,
    _experience_alignment,
    _keyword_overlap_ratio,
    _recommendation_for,
    score_resume,
    score_resume_quality,
)
from resume_service.storage import (
    DB_PATH,
    list_parsed_resumes,
    load_migration_run,
    load_parsed_resume,
    migrate_legacy_json_store,
    save_parsed_resume,
)
from resume_service.tests.fixture_builders import build_docx, build_legacy_doc_stub, build_pdf, load_

# ---------------------------------------------------------------------------
# API Contract Tests (via FastAPI TestClient)
# ---------------------------------------------------------------------------

class APIContractTests(unittest.TestCase):
    """Tests the full HTTP API surface including edge cases."""

    @classmethod
    def setUpClass(cls):
        cls.client = TestClient(app)

    # --- /v1/parse success paths ---

    def test_parse_pdf_returns_200(self):
        case = load_case("fresher_resume")
        resp = self.client.post(
            "/v1/parse",
            files={"file": ("r.pdf", build_pdf(case["lines"]), "application/pdf")},
            data={"resume_id": "res_api_pdf_001", "candidate_type": "fresher"},
        )
        self.assertEqual(resp.status_code, 200)
        body = resp.json()
        self.assertEqual(body["resume_id"], "res_api_pdf_001")

    def test_parse_docx_returns_200(self):
        case = load_case("lateral_resume")
        resp = self.client.post(
            "/v1/parse",
            files={"file": ("r.docx", build_docx(case["lines"]), "application/vnd.openxmlformats-offi
            data={"resume_id": "res_api_docx_001", "candidate_type": "lateral"},
        )
        self.assertEqual(resp.status_code, 200)

    def test_parse_default_candidate_type_is_lateral(self):
        case = load_case("fresher_resume")
        resp = self.client.post(
            "/v1/parse",
            files={"file": ("r.pdf", build_pdf(case["lines"]), "application/pdf")},
            data={"resume_id": "res_api_default_type"},
        )
        self.assertEqual(resp.status_code, 200)
        self.assertEqual(resp.json()["candidate_type"], "lateral")

    # --- /v1/parse rejection paths ---

    def test_parse_rejects_legacy_doc_via_api(self):
        resp = self.client.post(
            "/v1/parse",
            files={"file": ("r.doc", build_legacy_doc_stub(), "application/msword")},
            data={"resume_id": "res_api_legacy"},
        )
        # sniff.py raises UnsupportedResumeError which main.py maps to 422
```

```
        self.assertEqual(resp.status_code, 422)
        body = resp.json()
        self.assertIn("Legacy", body["message"])

    def test_parse_rejects_scanned_image_only_pdf_via_api(self):
        resp = self.client.post(
            "/v1/parse",
            files={"file": ("r.pdf", build_pdf([], image_only=True), "application/pdf")},
            data={"resume_id": "res_api_scanned"},
        )
        self.assertEqual(resp.status_code, 422)

    def test_parse_rejects_empty_file_field(self):
        """When file field is present but empty, the endpoint should reject."""
        resp = self.client.post(
            "/v1/parse",
            files={"file": ("", b"", "application/octet-stream")},
            data={"resume_id": "res_api_empty_file"},
        )
        # The multipart parser accepts the field but the filename is empty,
        # so the endpoint rejects it as invalid.
        self.assertIn(resp.status_code, {400, 422})

    def test_parse_rejects_missing_resume_id_field(self):
        case = load_case("fresher_resume")
        resp = self.client.post(
            "/v1/parse",
            files={"file": ("r.pdf", build_pdf(case["lines"]), "application/pdf")},
        )
        self.assertEqual(resp.status_code, 422)
        self.assertIn("resume_id", resp.json()["detail"].lower())

    # --- /v1/score: load from storage by resume_id only ---

    def test_score_by_resume_id_only(self):
        """Score endpoint should load stored parsed resume when only resume_id + job_context provide
        case = load_case("lateral_resume")
        # 1. Parse first
        self.client.post(
            "/v1/parse",
            files={"file": ("r.docx", build_docx(case["lines"]), "application/vnd.openxmlformats-offi
            data={"resume_id": case["resume_id"], "candidate_type": case["candidate_type"]},
        )
        # 2. Score with resume_id only (no normalized_resume, no resume_text)
        resp = self.client.post(
            "/v1/score",
            json={
                "resume_id": case["resume_id"],
                "job_context": {
                    "job_id": "job_001",
                    "title": "Backend Developer",
                    "required_skills": ["Java", "PostgreSQL"],
                },
            },
        )
        self.assertEqual(resp.status_code, 200)
        body = resp.json()
        self.assertIn("jd_match_score", body)
        self.assertIn("resume_quality_score", body)
        self.assertIn("recommendation", body)
        self.assertEqual(body["resume_id"], case["resume_id"])

    def test_score_returns_404_for_missing_resume_id(self):
        resp = self.client.post(
            "/v1/score",
            json={
                "resume_id": "does_not_exist_xyz",
                "job_context": {"job_id": "j1", "required_skills": []},
            },
        )
        self.assertEqual(resp.status_code, 404)

    # --- /v1/score: validation ---

    def test_score_rejects_partial_resume_inputs(self):
        """Providing normalized_resume without resume_text (or vice versa) should fail validation."""
```

```
        resp = self.client.post(
            "/v1/score",
            json={
                "resume_id": "res_x",
                "normalized_resume": {},
                "job_context": {"job_id": "j1", "required_skills": []},
            },
        )
        self.assertEqual(resp.status_code, 422)

    # --- Response schema shape ---

    def test_parse_response_shape(self):
        case = load_case("fresher_resume")
        resp = self.client.post(
            "/v1/parse",
            files={"file": ("r.pdf", build_pdf(case["lines"]), "application/pdf")},
            data={"resume_id": "res_shape_001"},
        )
        body = resp.json()
        for key in ("resume_id", "candidate_type", "normalized_resume", "warnings", "missing_fields",
            self.assertIn(key, body)
        rq = body["resume_quality"]
        for key in ("score", "breakdown", "recommendation", "summary"):
            self.assertIn(key, rq)

    def test_score_response_shape(self):
        case = load_case("lateral_resume")
        self.client.post(
            "/v1/parse",
            files={"file": ("r.docx", build_docx(case["lines"]), "application/vnd.openxmlformats-offi
            data={"resume_id": case["resume_id"], "candidate_type": case["candidate_type"]},
        )
        resp = self.client.post(
            "/v1/score",
            json={
                "resume_id": case["resume_id"],
                "job_context": {"job_id": "j1", "required_skills": ["Java"]},
            },
        )
        body = resp.json()
        for key in ("resume_id", "jd_match_score", "resume_quality_score", "breakdown", "matched_skil
            self.assertIn(key, body)
        self.assertIn("jd_match", body["breakdown"])
        self.assertIn("resume_quality", body["breakdown"])

    def test_metrics_endpoint_returns_counters(self):
        resp = self.client.get("/metrics")
        self.assertEqual(resp.status_code, 200)
        body = resp.json()
        self.assertIn("parse_success_count", body)
        self.assertIn("score_request_count", body)

    def test_resume_audit_endpoints_return_stored_data(self):
        case = load_case("fresher_resume")
        resume_id = "res_audit_001"
        parse_resp = self.client.post(
            "/v1/parse",
            files={"file": ("r.pdf", build_pdf(case["lines"]), "application/pdf")},
            data={"resume_id": resume_id, "candidate_type": "fresher"},
        )
        self.assertEqual(parse_resp.status_code, 200)

        list_resp = self.client.get("/v1/resumes?page=1&page_size=20")
        self.assertEqual(list_resp.status_code, 200)
        self.assertTrue(any(item["resume_id"] == resume_id for item in list_resp.json()["items"]))

        detail_resp = self.client.get(f"/v1/resumes/{resume_id}")
        self.assertEqual(detail_resp.status_code, 200)
        detail = detail_resp.json()
        self.assertEqual(detail["resume_id"], resume_id)
        self.assertIn("parser_metadata", detail)
        self.assertIn("normalized_resume", detail)

    # --- Swagger/example schema sanity ---
```

```
    def test_openapi_schema_is_valid(self):
        resp = self.client.get("/openapi.json")
        self.assertEqual(resp.status_code, 200)
        schema = resp.json()
        self.assertIn("/v1/parse", schema["paths"])
        self.assertIn("/v1/score", schema["paths"])
        self.assertIn("/health", schema["paths"])
        # Verify ScoreResumeRequest has expected properties
        schemas = schema["components"]["schemas"]
        score_req = schemas.get("ScoreResumeRequest", {})
        self.assertIn("properties", score_req)
        self.assertIn("resume_id", score_req["properties"])
        self.assertIn("job_context", score_req["properties"])

# ---------------------------------------------------------------------------
# Storage Tests
# ---------------------------------------------------------------------------

class StorageTests(unittest.TestCase):
    """Verify SQLite-backed resume storage behavior."""

    def test_save_and_load_roundtrip(self):
        case = load_case("fresher_resume")
        parsed = parse_resume_document(
            file_bytes=build_pdf(case["lines"]),
            filename="r.pdf",
            resume_id="res_storage_rt",
            candidate_type=CandidateType.fresher,
        )
        save_parsed_resume(parsed)
        loaded = load_parsed_resume("res_storage_rt")
        self.assertIsNotNone(loaded)
        self.assertEqual(loaded.resume_id, "res_storage_rt")
        self.assertEqual(loaded.normalized_resume.full_name, "Aarav Sharma")
        self.assertEqual(loaded.candidate_type, CandidateType.fresher)
        self.assertIn("aarav.sharma@email.com", loaded.normalized_resume.emails)

    def test_load_returns_none_for_missing_id(self):
        result = load_parsed_resume("this_id_never_existed")
        self.assertIsNone(result)

    def test_sqlite_database_exists_on_disk(self):
        case = load_case("lateral_resume")
        parsed = parse_resume_document(
            file_bytes=build_docx(case["lines"]),
            filename="r.docx",
            resume_id="res_disk_check",
            candidate_type=CandidateType.lateral,
        )
        save_parsed_resume(parsed)
        self.assertTrue(DB_PATH.exists())
        loaded = load_parsed_resume("res_disk_check")
        self.assertIsNotNone(loaded)
        self.assertEqual(loaded.resume_id, "res_disk_check")

    def test_list_parsed_resumes_returns_metadata(self):
        case = load_case("fresher_resume")
        parsed = parse_resume_document(
            file_bytes=build_pdf(case["lines"]),
            filename="r.pdf",
            resume_id="res_list_check",
            candidate_type=CandidateType.fresher,
        )
        save_parsed_resume(parsed)
        listing = list_parsed_resumes(page=1, page_size=20)
        self.assertGreaterEqual(listing.total, 1)
        self.assertTrue(any(item.resume_id == "res_list_check" for item in listing.items))

    def test_duplicate_fingerprint_is_linked_to_original_resume(self):
        case = load_case("fresher_resume")
        duplicate_lines = list(case["lines"]) + ["Fingerprint Marker: duplicate-test-2026"]
        first = parse_resume_document(
            file_bytes=build_pdf(duplicate_lines),
            filename="dup-a.pdf",
            resume_id="res_dup_fp_a",
            candidate_type=CandidateType.fresher,
```

```
            )
            second = parse_resume_document(
                file_bytes=build_pdf(duplicate_lines),
                filename="dup-b.pdf",
                resume_id="res_dup_fp_b",
                candidate_type=CandidateType.fresher,
            )

            save_parsed_resume(first)
            save_parsed_resume(second)

            stored_first = load_parsed_resume("res_dup_fp_a")
            stored_second = load_parsed_resume("res_dup_fp_b")
            self.assertIsNotNone(stored_first)
            self.assertIsNotNone(stored_second)
            self.assertIsNone(stored_first.duplicate_of_resume_id)
            self.assertEqual(stored_second.duplicate_of_resume_id, "res_dup_fp_a")

        def test_migration_run_persists_status_and_retirement(self):
            case = load_case("fresher_resume")
            parsed = parse_resume_document(
                file_bytes=build_pdf(case["lines"]),
                filename="legacy-import.pdf",
                resume_id="res_migration_source_001",
                candidate_type=CandidateType.fresher,
            )
            target_resume_id = f"res_migration_target_{uuid4().hex[:10]}"

            with tempfile.TemporaryDirectory() as tmp_dir:
                legacy_dir = Path(tmp_dir) / "resumes"
                legacy_dir.mkdir(parents=True, exist_ok=True)
                payload = {
                    "resume_id": target_resume_id,
                    "candidate_type": "fresher",
                    "normalized_resume": parsed.normalized_resume.model_dump(),
                    "warnings": parsed.warnings,
                    "missing_fields": parsed.missing_fields,
                    "resume_quality": parsed.resume_quality.model_dump(),
                    "parser_metadata": parsed.parser_metadata.model_dump(),
                }
                source_file = legacy_dir / "legacy_test_resume.json"
                source_file.write_text(json.dumps(payload), encoding="utf-8")

                with mock.patch("resume_service.storage.RESUME_STORE_DIR", legacy_dir):
                    result = migrate_legacy_json_store(retire_legacy_files=True)

                self.assertEqual(result["status"], "success")
                self.assertEqual(result["imported_count"], 1)
                self.assertEqual(result["retired_count"], 1)
                self.assertFalse(source_file.exists())

                retired_file = legacy_dir.parent / "resumes_retired" / result["run_id"] / source_file.nam
                self.assertTrue(retired_file.exists())

                run = load_migration_run(result["run_id"])
                self.assertIsNotNone(run)
                self.assertEqual(run["status"], "success")
                self.assertEqual(run["imported_count"], 1)

    # ---------------------------------------------------------------------------
    # Phone Normalization Tests
    # ---------------------------------------------------------------------------

    class PhoneNormalizationTests(unittest.TestCase):
        def test_10_digit_indian_number(self):
            self.assertEqual(_normalize_phone("9876543210"), "+919876543210")

        def test_10_digit_with_spaces(self):
            self.assertEqual(_normalize_phone("98765 43210"), "+919876543210")

        def test_10_digit_with_dashes(self):
            self.assertEqual(_normalize_phone("98765-43210"), "+919876543210")

        def test_12_digit_with_91_prefix(self):
            self.assertEqual(_normalize_phone("919876543210"), "+919876543210")
```

```
    def test_12_digit_with_plus_91(self):
        self.assertEqual(_normalize_phone("+919876543210"), "+919876543210")

    def test_11_digit_with_leading_zero(self):
        self.assertEqual(_normalize_phone("09876543210"), "+919876543210")

    def test_invalid_number_too_short(self):
        self.assertIsNone(_normalize_phone("12345"))

    def test_invalid_number_starts_with_wrong_digit(self):
        self.assertIsNone(_normalize_phone("1234567890"))

    def test_number_from_resume_fresher(self):
        case = load_case("fresher_resume")
        raw_text = "\n".join(case["lines"])
        parsed = parse_resume_document(
            file_bytes=build_pdf(case["lines"]),
            filename="r.pdf",
            resume_id="res_phone_fresher",
            candidate_type=CandidateType.fresher,
        )
        self.assertIn("+919876543210", parsed.normalized_resume.phones)

# ---------------------------------------------------------------------------
# Education Normalization Tests
# ---------------------------------------------------------------------------

class EducationNormalizationTests(unittest.TestCase):
    def test_btech_education_parsed(self):
        case = load_case("fresher_resume")
        parsed = parse_resume_document(
            file_bytes=build_pdf(case["lines"]),
            filename="r.pdf",
            resume_id="res_edu_001",
            candidate_type=CandidateType.fresher,
        )
        education = parsed.normalized_resume.education
        self.assertTrue(education)
        self.assertEqual(education[0].degree, "B.Tech")
        self.assertEqual(education[0].completion_year, 2025)

    def test_be_education_parsed(self):
        case = load_case("lateral_resume")
        parsed = parse_resume_document(
            file_bytes=build_docx(case["lines"]),
            filename="r.docx",
            resume_id="res_edu_002",
            candidate_type=CandidateType.lateral,
        )
        degrees = [e.degree for e in parsed.normalized_resume.education]
        self.assertIn("B.E", degrees)

    def test_cgpa_extracted(self):
        case = load_case("fresher_resume")
        parsed = parse_resume_document(
            file_bytes=build_pdf(case["lines"]),
            filename="r.pdf",
            resume_id="res_edu_003",
            candidate_type=CandidateType.fresher,
        )
        edu = parsed.normalized_resume.education[0]
        self.assertIsNotNone(edu.score)
        self.assertIn("8.4", edu.score)

# ---------------------------------------------------------------------------
# Section Detection Tests
# ---------------------------------------------------------------------------

class SectionDetectionTests(unittest.TestCase):
    def test_all_section_aliases_match(self):
        """Verify that all SECTION_ALIASES keys have at least one alias."""
        from resume_service.constants import SECTION_ALIASES
        for section, aliases in SECTION_ALIASES.items():
            self.assertTrue(len(aliases) > 0, f"Section {section} has no aliases")

    def test_sections_split_correctly(self):
```

```
            lines = [
                "John Doe",
                "john@email.com",
                "SUMMARY",
                "Experienced developer.",
                "TECHNICAL SKILLS",
                "Python, Java",
                "EXPERIENCE",
                "Senior Dev | Google | Jan 2020 - Present",
                "Did things.",
                "EDUCATION",
                "B.Tech in CS | MIT | 2019",
            ]
            cleaned = [l.replace("\u200b", "").strip() for l in lines if l.strip()]
            sections = _split_sections(cleaned)
            self.assertIn("summary", sections)
            self.assertIn("skills", sections)
            self.assertIn("experience", sections)
            self.assertIn("education", sections)
            # Content should be under correct section
            self.assertTrue(any("Experienced developer" in l for l in sections["summary"]))
            self.assertTrue(any("Python" in l for l in sections["skills"]))

        def test_unknown_heading_keeps_previous_section(self):
            lines = [
                "SUMMARY",
                "Good candidate.",
                "RANDOM SECTION",
                "Some text.",
            ]
            sections = _split_sections(lines)
            self.assertIn("summary", sections)
            # "Some text." should be in "summary" since "RANDOM SECTION" doesn't match
            all_text = " ".join(sections.get("summary", []))
            self.assertIn("Some text.", all_text)

    # ---------------------------------------------------------------------------
    # Sensitive Field Filtering Tests
    # ---------------------------------------------------------------------------

    class SensitiveFieldFilteringTests(unittest.TestCase):
        def test_pan_detected_in_raw_text(self):
            from resume_service.normalization import _detect_sensitive_findings
            text = "John Doe\nPAN: ABCDE1234F\nEmail: john@email.com"
            findings = _detect_sensitive_findings(text)
            self.assertIn("pan", findings)

        def test_aadhaar_detected_in_raw_text(self):
            from resume_service.normalization import _detect_sensitive_findings
            text = "Aadhaar: 1234 5678 9012"
            findings = _detect_sensitive_findings(text)
            self.assertIn("aadhaar", findings)

        def test_sensitive_findings_produce_warning(self):
            case = load_case("lateral_resume")
            parsed = parse_resume_document(
                file_bytes=build_docx(case["lines"]),
                filename="r.docx",
                resume_id="res_sensitive_001",
                candidate_type=CandidateType.lateral,
            )
            self.assertIn("pan", parsed.parser_metadata.sensitive_findings)
            self.assertIn("aadhaar", parsed.parser_metadata.sensitive_findings)
            # Warning should be raised
            self.assertTrue(any("Sensitive" in w for w in parsed.warnings))

        def test_no_sensitive_fields_in_normalized_resume(self):
            """Sensitive fields should NOT appear in NormalizedResume fields."""
            case = load_case("lateral_resume")
            parsed = parse_resume_document(
                file_bytes=build_docx(case["lines"]),
                filename="r.docx",
                resume_id="res_sensitive_002",
                candidate_type=CandidateType.lateral,
            )
            nr = parsed.normalized_resume
            # PAN and Aadhaar patterns should not appear in any normalized field
```

```
        combined = json.dumps(nr.model_dump())
        self.assertNotIn("ABCDE1234F", combined)
        self.assertNotIn("1234 5678 9012", combined)

# ---------------------------------------------------------------------------
# Scoring Tests
# ---------------------------------------------------------------------------

class ScoringDetailedTests(unittest.TestCase):
    def test_deterministic_score_output_shape(self):
        """Score output should always have consistent shape with integer scores."""
        case = load_case("lateral_resume")
        parsed = parse_resume_document(
            file_bytes=build_docx(case["lines"]),
            filename="r.docx",
            resume_id="res_score_shape_001",
            candidate_type=CandidateType.lateral,
        )
        resp = score_resume(
            ScoreResumeRequest(
                resume_id=parsed.resume_id,
                normalized_resume=parsed.normalized_resume,
                resume_text=parsed.parser_metadata.extracted_text,
                job_context=JobContext(
                    job_id="j1",
                    title="Developer",
                    required_skills=["Java"],
                ),
            )
        )
        self.assertIsInstance(resp.jd_match_score, int)
        self.assertIsInstance(resp.resume_quality_score, int)
        self.assertIsInstance(resp.breakdown.jd_match, dict)
        self.assertIsInstance(resp.breakdown.resume_quality, dict)
        self.assertIn(resp.recommendation, {"SHORTLIST", "REVIEW", "REJECT"})

    def test_recommendation_shortlist(self):
        self.assertEqual(_recommendation_for(85, 60), "SHORTLIST")

    def test_recommendation_review_by_jd(self):
        self.assertEqual(_recommendation_for(65, 40), "REVIEW")

    def test_recommendation_review_by_quality(self):
        self.assertEqual(_recommendation_for(50, 65), "REVIEW")

    def test_recommendation_reject(self):
        self.assertEqual(_recommendation_for(30, 30), "REJECT")

    def test_experience_alignment_in_range(self):
        job = JobContext(experience_min_years=2, experience_max_years=5)
        # Exactly in range
        self.assertEqual(_experience_alignment(3.0, job), 1.0)
        # Below minimum
        self.assertGreaterEqual(_experience_alignment(1.0, job), 0.2)
        self.assertLessEqual(_experience_alignment(1.0, job), 1.0)
        # Above maximum
        self.assertGreaterEqual(_experience_alignment(8.0, job), 0.75)

    def test_experience_alignment_no_bounds(self):
        job = JobContext()
        self.assertEqual(_experience_alignment(5.0, job), 1.0)
        self.assertEqual(_experience_alignment(0.0, job), 0.5)

    def test_keyword_overlap_ratio(... [truncated]
```

▼ Show more

Thinking

I now have a comprehensive understanding of the entire service. Here is the complete analysis:

Complete Analysis of d:\Portal\resume_service

1. API ENDPOINTS

The service is a FastAPI application (main.py) exposing these endpoints:

GET /health

Purpose: Liveness probe

Response: {"status": "ok"}

GET /metrics
Purpose: Operational metrics counters
Response: MetricsResponse model with fields:
parse_success_count, parse_failure_count, score_request_count
unsupported_format_reject_count, scanned_resume_reject_count
average_parse_latency_ms, average_score_latency_ms
POST /v1/parse
Content-Type: multipart/form-data
Request fields:
file (binary, required) – PDF or DOCX resume
resume_id (string, required) – identifier from the main backend (e.g. "res_123")
candidate_type (string, optional, default "lateral") – "fresher" or "lateral"
Response (ParseResumeResponse):

```
{
  "resume_id": "res_123",
  "candidate_type": "lateral",
  "normalized_resume": { ... },
  "warnings": ["Sensitive personal identifiers were detected..."],
  "missing_fields": ["full_name", "skills"],
  "resume_quality": { "score": 98, "breakdown": {...}, "recommendation": "STRONG", "summary": "..." }
  "parser_metadata": { "filename": "...", "file_type": "pdf", "content_type": "...", "file_size_bytes
}
```

Errors: 400 (bad multipart/extraction), 422 (unsupported format, scanned/image PDF, legacy .doc, miss
POST /v1/score
Content-Type: application/json
Request (ScoreResumeRequest):

```
{
  "resume_id": "res_123",
  "normalized_resume": null,
  "resume_text": null,
  "job_context": {
    "job_id": "job_001",
    "title": "Backend Developer",
    "description": "...",
    "required_skills": ["Python", "SQL"],
    "preferred_skills": ["Flask", "Docker"],
    "keywords": ["backend", "api"],
    "experience_min_years": 0,
    "experience_max_years": 2,
    "location": "Hyderabad"
  }
}
```

If normalized_resume and resume_text are both null, the service loads parsed data from SQLite storage
If provided, both must be supplied together (validated by model_validator).
Response (ScoreResumeResponse):

```
{
  "resume_id": "res_123",
  "jd_match_score": 85,
  "resume_quality_score": 72,
  "breakdown": {
    "jd_match": { "skills": 42.5, "experience": 30.0, "keywords": 12.5 },
    "resume_quality": { "completeness": 32.0, "structure": 20.0, "contactability": 16.0, "signal_qua
  },
  "matched_skills": ["Python", "SQL"],
  "missing_skills": ["Git"],
  "recommendation": "SHORTLIST",
  "warnings": ["Some required job skills are missing..."],
  "summary": "JD match 85/100 and resume quality 72/100. Strongest matches: Python, SQL. Missing: Gi
}
```

GET /v1/resumes
Query params: page (int, default 1), page_size (int, default 20, max 100)
Response (ResumeListResponse): paginated list of ResumeListItem with metadata (resume_id, candidate_
GET /v1/resumes/{resume_id}
Response: Full stored resume document including normalized_resume, resume_quality, parser_metadata, r
Error: 404 if not found
2. RESUME PARSING AND AI SCORING PIPELINE
The pipeline flows through these stages:

Stage 1: Extraction (extractors/)
sniff.py – Dispatcher. Inspects file bytes:
b"%PDF" -> extract_pdf() (pdf.py)
b"PK" (ZIP) -> checks for DOCX structure -> extract_docx() (docx.py)
Legacy D0CF11E0 (.doc) -> raises UnsupportedResumeError
pdf.py – Two-tier PDF extraction:
pypdf (primary) – uses pypdf.PdfReader for standard text extraction
Custom layout parser (fallback) – raw PDF object parsing: parses PDF objects, builds font maps from T
docx.py – Extracts text from DOCX by parsing word/document.xml XML within the ZIP archive. Handles pa

base.py – Defines ExtractionResult dataclass (file_type, text, lines, content_type, detected_columns,
Stage 2: Normalization (normalization.py)
Takes raw lines and text, produces a NormalizedResume model:

Section splitting – Matches lines against SECTION_ALIASES (summary, skills, experience, internships,
Contact info – Regex extraction of emails, Indian phone numbers (normalizes to +91XXXXXXXXXX), URLs/
Name extraction – Heuristic scoring from header lines (title case, uppercase, position)
Location – Detects Indian cities via INDIAN_LOCATIONS mapping, normalizes to "City, State" format
Skills – Matches against SKILL_ALIASES dictionary (60+ skills with aliases). Builds SkillEvidenceRec
Education – Matches against DEGREE_ALIASES (B.Tech, M.Tech, MBA, etc.), extracts institution, specia
Employment history – Parses pipe-delimited format (Title | Company | Dates | Location) and free-form
Projects – Extracts project titles, tech stacks (via skill matching), summaries
Certifications – Parses certification names, issuers, dates
CTC/Notice Period/Work Auth/Preferred Location – Regex pattern matching
Sensitive data detection – Detects Aadhaar, PAN, passport, family details, religion, caste, marital s
Experience calculation – full_time_months, internship_months, total_relevant_months (internships weig
Stage 3: Quality Scoring (scoring.py)
score_resume_quality() produces a 0-100 score:

Completeness (40 pts max) – 10 boolean checks (name, email, phone, location, skills, education, curre
Structure (25 pts max) – Ratios for experience_summary (0.25), skills (0.2), education (0.2), employm
Contactability (20 pts max) – emails (8), phones (8), links (2), location (2)
Signal Quality (15 pts max) – text length (6), skills count (4), employment/education (3), projects
Recommendations: STRONG (>=75), REVIEW (>=55), IMPROVE (<55)
Stage 4: JD Match Scoring (scoring.py)
score_resume() produces a 0-100 JD match score:

Skills (50 pts) – Required skills coverage (85% weight) + preferred skills coverage (15% weight), wei
Experience (30 pts) – Alignment of candidate experience against job min/max years using _experience_a
Keywords (20 pts) – Overlap ratio of job keywords (from title, description, keywords list) against re
Recommendations: SHORTLIST (JD>=80 AND quality>=55), REVIEW (JD>=60 OR quality>=60), REJECT otherwise
3. DATABASE MODELS AND SCHEMAS
SQLite Database: d:\Portal\resume_service\data\resume_catalog.sqlite3
Table: parsed_resumes

resume_id TEXT PRIMARY KEY
candidate_type TEXT NOT NULL (enum: “fresher” or “lateral”)
normalized_resume_json TEXT NOT NULL (full JSON blob)
parser_metadata_json TEXT NOT NULL (full JSON blob)
resume_quality_json TEXT NOT NULL (full JSON blob)
resume_text TEXT NOT NULL (raw extracted text)
filename TEXT
full_name TEXT (denormalized for listing)
current_role TEXT (denormalized for listing)
content_fingerprint TEXT (SHA-256 hash for duplicate detection)
duplicate_of_resume_id TEXT (nullable FK to another resume_id)
created_at TEXT NOT NULL (ISO 8601 UTC)
updated_at TEXT NOT NULL (ISO 8601 UTC)
Table: migration_runs

run_id TEXT PRIMARY KEY
started_at TEXT NOT NULL
finished_at TEXT
status TEXT NOT NULL
source_dir TEXT NOT NULL
retire_legacy_files INTEGER NOT NULL DEFAULT 0
files_discovered INTEGER NOT NULL DEFAULT 0
imported_count, skipped_count, failed_count, retired_count (all INTEGER)
error_message TEXT
Legacy JSON Store: d:\Portal\resume_service\data\resumes*.json
Individual JSON files per resume (legacy format). Migration tool (migrate_json_store.py) imports thes

Data JSON files: d:\Portal\resume_service\data\parsed_resumes.json
Contains pre-parsed sample resumes: res_fresher_001 (Aarav Sharma), res_lateral_001 (Priya Nair), res

4. DATA FLOW
Client -> POST /v1/parse (multipart)
    |
    v
parse_multipart() --> MultipartPart {file, resume_id, candidate_type}
    |
    v
parse_resume_document()
    |
    +-> extract_resume() [sniff.py] --> ExtractionResult {text, lines, file_type, ...}
    | |

```
    |       +-> extract_pdf() [pdf.py] (pypdf first, custom layout parser fallback)
    |       +-> extract_docx() [docx.py] (XML parsing from ZIP)
    |
    +-> normalize_resume() [normalization.py] --> NormalizedResume + warnings + missing_fields
    |
    +-> score_resume_quality() [scoring.py] --> ResumeQualityResult {score, breakdown, recommendation
    |
    +-> ParserMetadata (filename, fingerprints, sensitive_findings, etc.)
    |
    v
ParseResumeResponse --> save_parsed_resume() [storage.py] --> SQLite
    |
    v
JSON response to client

Client -> POST /v1/score (JSON)
    |
    +-> If no normalized_resume/resume_text: load_parsed_resume(resume_id) from SQLite
    |
    +-> score_resume() [scoring.py]
    |       +-> _canonicalize_skills() (alias resolution via SKILL_ALIASES)
    |       +-> _skill_evidence_strength() (evidence weighting)
    |       +-> _experience_alignment() (min/max year comparison)
    |       +-> _keyword_overlap_ratio() (text search)
    |       +-> score_resume_quality() (recomputed)
    |
    v
ScoreResumeResponse to client
5. CONFIGURATION AND CONSTANTS
settings.py – AppSettings dataclass:

max_file_size_bytes: 5 MB
supported_extensions: .pdf, .docx
blocked_extensions: .doc
minimum_text_characters: 60
rate_limit_requests_per_window: 30 per 60 seconds
jd_score_weights: skills=50, experience=30, keywords=20
quality_score_weights: completeness=40, structure=25, contactability=20, signal_quality=15
constants.py – Lookup dictionaries:

SECTION_ALIASES: 8 section types with multiple name variants each
DEGREE_ALIASES: 14 degree types with common abbreviations
SKILL_ALIASES: 60+ tech skills with synonym mappings
INDIAN_LOCATIONS: 24 Indian cities normalized to "City, State" format
SENSITIVE_PATTERNS: 8 regex patterns (Aadhaar, PAN, passport, family_details, religion, caste, marita
NOTICE_PERIOD_PATTERNS, CTC_PATTERNS, WORK_AUTHORIZATION_PATTERNS, PREFERRED_LOCATION_PATTERNS: regex
MONTH_NAMES: all 12 months with abbreviations
6. MAIN ENTRY POINTS AND ROUTING
FastAPI app defined in main.py with app = FastAPI(title="Resume Service", version="0.1.0")
Startup event: init_storage() initializes SQLite database and creates tables
Middleware: request_context_middleware adds x-request-id header, enforces rate limiting on /v1/parse
Exception handlers: Custom handlers for MultipartError (400), UnsupportedResumeError (422), Extractio
To run: uvicorn resume_service.main:app (per pyproject.toml dependencies)
Migration CLI: python -m resume_service.migrate_json_store [--retire-legacy-files]
7. DATA STRUCTURES
Core Models (Pydantic, in schemas.py):
NormalizedResume – The central candidate profile:

full_name, emails[], phones[], current_location, links[]
skills[], skill_evidence[] (SkillEvidenceRecord: skill, evidence_type, source_section)
experience_summary, employment_history[] (EmploymentRecord: company, title, start_date, end_date, emp
education[] (EducationRecord: degree, specialization, institution, completion_year, score)
projects[] (ProjectRecord: title, tech_stack[], summary, source_section)
certifications[] (CertificationRecord: name, issuer, issued_date)
current_role, full_time_months, internship_months, total_relevant_months
notice_period, current_ctc, expected_ctc, preferred_location, work_authorization
CandidateType: Enum "fresher" | "lateral"

JobContext: job_id, title, description, required_skills[], preferred_skills[], keywords[], experience

ParserMetadata: filename, file_type, content_type, file_size_bytes, line_count, page_count, extractor

ParseResumeResponse: resume_id, candidate_type, normalized_resume, warnings[], missing_fields[], resu

ScoreResumeResponse: resume_id, jd_match_score (int 0-100), resume_quality_score (int 0-100), breakdo
```

8. INTEGRATION POINTS WITH FRONTEND
The service is designed as an internal microservice. Integration points:

Resume Upload: The frontend (or main backend) sends a multipart POST to /v1/parse with a file, resume

Scoring: After parsing, the frontend (via backend) calls /v1/score with just resume_id + job_context

Resume Listing/Audit: GET /v1/resumes (paginated) and GET /v1/resumes/{resume_id} provide resume brow

Health/Metrics: GET /health and GET /metrics for infrastructure monitoring.

Request Tracing: The service respects x-request-id headers for distributed tracing and returns it on

No direct frontend integration found – the service has no CORS configuration, no websockets, no HTML

ReadFile

jobs.csv
id,title,department,location,status,applicants,newToday,postedDate,timeToHireDays
1,Senior UX Designer,Design,"San Francisco, CA",Active,128,12,2026-03-29,21
2,Product Manager,Product,Remote,Active,84,4,2026-03-24,19
3,Lead Backend Engineer,Engineering,"New York, NY",Draft,0,0,,
4,Growth Marketing Lead,Marketing,"London, UK",Closed,246,0,2026-02-10,27
5,Data Analyst,Data,Remote,Active,61,3,2026-04-02,16
6,Talent Operations Partner,People,"Austin, TX",Closed,39,0,2026-01-18,14
▼ Show more

Thinking
Now let me read a few more critical files to understand the complete picture:

ReadFile

applications.csv
id,name,email,phone,role,match,status,date,avatarSeed
1,Elena Soros,elena.s@example.com,+1 (555) 012-3456,Senior UX Designer,94,Shortlisted,2026-04-07,ele
2,Marcus Johnson,marcus.j@example.com,+1 (555) 012-7890,Senior UX Designer,78,Pending,2026-04-06,mar
3,Sarah Chen,s.chen@corp.com,+1 (555) 012-9988,Product Manager,42,Rejected,2026-04-05,sarah-chen
4,Aarav Sharma,aarav.sharma@email.com,+91 98765 43210,Lead Backend Engineer,88,Shortlisted,2026-04-0
5,Priya Nair,priya.nair@sample.com,+91 99887 66554,Lead Backend Engineer,91,Shortlisted,2026-04-06,pr
6,Nina Patel,nina.patel@example.com,+1 (555) 017-2144,Data Analyst,81,Pending,2026-04-04,nina-patel
7,Daniel Brooks,daniel.brooks@example.com,+1 (555) 017-8832,Growth Marketing Lead,67,Rejected,2026-04
8,Meera Joshi,meera.joshi@example.com,+91 98877 66554,Talent Operations Partner,85,Pending,2026-04-02
9,Olivia Reed,olivia.reed@example.com,+1 (555) 018-4421,Product Manager,89,Shortlisted,2026-04-01,oli
10,Vikram Rao,vikram.rao@example.com,+91 90000 11223,Data Analyst,76,Pending,2026-03-31,vikram-rao
▼ Show more
ReadFile

API_ENDPOINTS.md
Showing lines 1-100 of 1534 total lines.

Coastal Careers -- API Endpoint Reference

> AI-powered career matching platform. All endpoints use `/api/v1` prefix. Requests require `Content-

---

Table of Contents

---

1. Authentication

1.1 Register

Field	Value
Method	`POST`
URL	`/api/v1/auth/register`
Description	Create a new candidate or recruiter account

Request Body Schema

```json
{
  "email": "string",
  "password": "string",
  "first_name": "string",
  "last_name": "string",
  "role": "\"candidate\" | \"recruiter\" | \"admin\"",
  "phone": "string (optional)"
}
```

Example Test Payload

```json
{
  "email": "sarah.johnson@email.com",
  "password": "Str0ng!Passw0rd",
  "first_name": "Sarah",
  "last_name": "Johnson",
  "role": "candidate",
  "phone": "+1 (415) 555-0142"
}
```

1.2 Login

Field	Value
Method	`POST`
URL	`/api/v1/auth/login`
Description	Authenticate and receive access + refresh tokens

Request Body Schema

```json
{
  "email": "string",
  "password": "string"
}
```

Example Test Payload

```json
{
  "email": "sarah.johnson@email.com",
  "password": "Str0ng!Passw0rd"
}
```

1.3 Refresh Token

Field	Value
Method	`POST`
URL	`/api/v1/auth/refresh`
Description	Obtain a new access token using a valid refresh token

Request Body Schema

```json
{
```

```
  "refresh_token": "string"
}
```

▼ Show more
Search
Glob: 'd:\Portal\frontend\adminPages\src***'
■
1 line of output

Thinking
Search
ListFiles: frontend\adminPages
■
13 lines of output

Thinking
ReadFile

server.ts

```
import express from "express";
import { createServer as createViteServer } from "vite";
import path from "path";
import { fileURLToPath } from "url";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

async function startServer() {
  const app = express();
  const PORT = 3000;

  app.use(express.json());

  // --- API ENDPOINTS (Placeholder Logic) ---

  // User Management
  app.get("/api/users", (req, res) => {
    // TODO: Fetch users from database
    res.json({ message: "GET users endpoint", data: [] });
  });

  app.post("/api/users", (req, res) => {
    const userData = req.body;
    // TODO: Create user in database
    res.status(201).json({ message: "User created", data: userData });
  });

  app.patch("/api/users/:id", (req, res) => {
    const { id } = req.params;
    const updates = req.body;
    // TODO: Update user in database
    res.json({ message: `User ${id} updated`, data: updates });
  });

  app.delete("/api/users/:id", (req, res) => {
    const { id } = req.params;
    // TODO: Delete user from database
    res.json({ message: `User ${id} deleted` });
  });

  // Job Management
  app.get("/api/jobs", (req, res) => {
    // TODO: Fetch jobs from database
    res.json({ message: "GET jobs endpoint", data: [] });
  });

  app.post("/api/jobs", (req, res) => {
    const jobData = req.body;
    // TODO: Create job in database
    res.status(201).json({ message: "Job created", data: jobData });
  });

  app.patch("/api/jobs/:id", (req, res) => {
    const { id } = req.params;
    const updates = req.body;
    // TODO: Update job in database
```

```
    res.json({ message: `Job ${id} updated`, data: updates });
  });

  app.delete("/api/jobs/:id", (req, res) => {
    const { id } = req.params;
    // TODO: Delete job from database
    res.json({ message: `Job ${id} deleted` });
  });

  // Email Automations
  app.post("/api/automations/email", (req, res) => {
    const { type, template } = req.body;
    // TODO: Update email template
    res.json({ message: `${type} template updated`, template });
  });

  // --- VITE MIDDLEWARE ---

  if (process.env.NODE_ENV !== "production") {
    const vite = await createViteServer({
      server: { middlewareMode: true },
      appType: "spa",
    });
    app.use(vite.middlewares);
  } else {
    const distPath = path.join(process.cwd(), "dist");
    app.use(express.static(distPath));
    app.get("*", (req, res) => {
      res.sendFile(path.join(distPath, "index.html"));
    });
  }

  app.listen(PORT, "0.0.0.0", () => {
    console.log(`Server running on http://localhost:${PORT}`);
  });
}

startServer();
```
▼ Show more
ReadFile

server.ts
```
import express from "express";
import { createServer as createViteServer } from "vite";
import path from "path";
import { fileURLToPath } from "url";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

async function startServer() {
  const app = express();
  const PORT = 3000;

  app.use(express.json());

  // --- JOB ENDPOINTS ---

  // List all jobs
  app.get("/api/jobs", (req, res) => {
    // TODO: Implement database fetch
    res.json({ jobs: [] });
  });

  // Create a new job
  app.post("/api/jobs", (req, res) => {
    // TODO: Implement database insert
    res.status(201).json({ message: "Job created successfully", data: req.body });
  });

  // Get single job
  app.get("/api/jobs/:id", (req, res) => {
    // TODO: Implement database fetch by ID
    res.json({ job: null });
  });
```

```
  // Update job
  app.put("/api/jobs/:id", (req, res) => {
    // TODO: Implement database update
    res.json({ message: "Job updated successfully", id: req.params.id });
  });

  // Delete job
  app.delete("/api/jobs/:id", (req, res) => {
    // TODO: Implement database delete
    res.json({ message: "Job deleted successfully" });
  });

  // --- APPLICATION ENDPOINTS ---

  // List all applications
  app.get("/api/applications", (req, res) => {
    // TODO: Implement database fetch
    res.json({ applications: [] });
  });

  // Submit application
  app.post("/api/applications", (req, res) => {
    // TODO: Implement database insert
    res.status(201).json({ message: "Application submitted", data: req.body });
  });

  // Get application details
  app.get("/api/applications/:id", (req, res) => {
    // TODO: Implement database fetch
    res.json({ application: null });
  });

  // Update application status (Shortlist/Reject)
  app.put("/api/applications/:id", (req, res) => {
    // TODO: Implement status update
    res.json({ message: "Application status updated", id: req.params.id });
  });

  // --- BULK UPLOAD ---

  app.post("/api/bulk-upload", (req, res) => {
    // TODO: Implement file processing logic
    res.json({ message: "Bulk upload started", count: 0 });
  });

  // --- VITE MIDDLEWARE ---

  if (process.env.NODE_ENV !== "production") {
    const vite = await createViteServer({
      server: { middlewareMode: true },
      appType: "spa",
    });
    app.use(vite.middlewares);
  } else {
    const distPath = path.join(process.cwd(), "dist");
    app.use(express.static(distPath));
    app.get("*", (req, res) => {
      res.sendFile(path.join(distPath, "index.html"));
    });
  }

  app.listen(PORT, "0.0.0.0", () => {
    console.log(`Server running on http://localhost:${PORT}`);
  });
}

startServer();
▼ Show more
ReadFile

App.tsx
Showing lines 1-100 of 696 total lines.

---
```

```
/**
 * @license
 * SPDX-License-Identifier: Apache-2.0
 */

import React, { useState } from "react";
import { motion, AnimatePresence } from "motion/react";
import {
  Bell,
  Settings,
  Plus,
  MoreVertical,
  Eye,
  Edit2,
  CircleSlash,
  Mail,
  CheckCircle2,
  TrendingUp,
  Info,
  UserPlus,
  Search,
  ChevronDown
} from "lucide-react";

type View = 'portal' | 'add-hr';

interface User {
  id: string;
  name: string;
  email: string;
  role: string;
  status: 'Active' | 'Disabled' | 'Inactive';
  initials: string;
  color?: string;
}

const TopNav = ({ currentView, setView }: { currentView: View, setView: (v: View) => void }) => (
  <nav className="fixed top-0 w-full z-50 glass shadow-luminous">
    <div className="flex justify-between items-center px-8 h-16 w-full max-w-screen-2xl mx-auto">
      <div className="flex items-center gap-8">
        <button
          onClick={() => setView('portal')}
          className="text-xl font-bold bg-gradient-to-br from-primary to-teal-accent bg-clip-text tex
        >
          Coastal Seven
        </button>
        <div className="hidden md:flex items-center gap-6">
          {["Home", "Applications", "Dashboard"].map((item) => (
            <a key={item} href="#" className="text-on-surface-variant hover:text-primary transition-c
              {item}
            </a>
          ))}
          <button
            onClick={() => setView('portal')}
            className={`text-sm font-medium pb-1 transition-all ${currentView === 'portal' ? 'text-pr
          >
            Admin
          </button>
        </div>
      </div>
      <div className="flex items-center gap-4">
        <button className="p-2 hover:bg-surface-container-low rounded-full transition-all">
          <Bell className="w-5 h-5 text-on-surface-variant" />
        </button>
        <button className="p-2 hover:bg-surface-container-low rounded-full transition-all">
          <Settings className="w-5 h-5 text-on-surface-variant" />
        </button>
        <div className="w-8 h-8 rounded-full overflow-hidden border border-on-surface-variant/30">
          <img
            src="https://picsum.photos/seed/profile/100/100"
            alt="Profile"
            referrerPolicy="no-referrer"
            className="w-full h-full object-cover"
          />
        </div>
      </div>
```

```
    </div>
  </nav>
);

const AdminPortal: React.FC<{
  onAddHR: () => void,
  users: User[],
  onToggleStatus: (id: string) => void,
  onDelete: (id: string) => void,
  onEdit: (user: User) => void,
  onView: (user: User) => void
}> = ({ onAddHR, users, onToggleStatus, onDelete, onEdit, onView }) => (
  <motion.div
    initial={{ opacity: 0, x: -20 }}
    animate={{ opacity: 1, x: 0 }}
    exit={{ opacity: 0, x: 20 }}
    className="w-full"
  >
    <header className="mb-12 pt-24">
      <h1 className="text-[3.5rem] font-bold tracking-tight text-on-surface leading-none mb-4">
        Admin Portal
      </h1>
      <p className="text-on-surface-variant text-lg max-w-2xl">
        Luminous command center for system-level precision. Manage core infrastructure, job definitio
▼ Show more

Thinking
Perfect! Now I have a complete understanding of the entire repository. Let me create the comprehensiv
```